

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Mutation Testing Effectiveness

An Empirical Analysis on Java Mutation Testing Tools

Luís Carlos de Sousa Pinto



Mestrado em Engenharia de Software

Supervisor: Prof. Ana Paiva

Co-Supervisor: Prof. João Bispo

October 31, 2022

Mutation Testing Effectiveness
An Empirical Analysis on Java Mutation Testing Tools

Luís Carlos de Sousa Pinto

Mestrado em Engenharia de Software

Approved in oral examination by the committee:

Chair: Prof. Miguel Brito

Supervisor: Prof. Ana Paiva

Co-Supervisor: Prof. João Bispo

October 31, 2022

Abstract

Software testing is an integral component of software development. Testing methods and paradigms are no longer merely a passing fad in software development models. Unit testing, which enables programmers to test each component separately, is one of the most widely used methodologies today and is required in the majority of software development companies. Test-Driven Development (TDD) goes even beyond that by coding tests based on supplied requirements before writing the real program code.

When these methods are examined more closely, it becomes clear that significant effort is put into the testing part of the software development process. Studies have shown, that software testing can cost up to 50% of development budget [27]. As a direct result of this, test suits need to be efficient and dependable in locating software faults, which will ultimately result in software that has a higher quality. But how can a test suite be accurately evaluated, and how can developers have confidence that their code will not fail or provide unexpected results? That's where Mutation Testing comes in.

Since it was initially introduced in the 1970s [2], mutation testing has been the subject of an ever-increasing and comprehensive body of research. Mutation testing is a structural, fault-based technique that is used to verify the efficiency or efficacy of a test suite. It is useful for determining the appropriateness of test suites and for determining the amount of faults that exist in an application that is currently being evaluated. The main idea behind Mutation testing method is to generate a large number of variant copies of the original program known as mutants. The test suite is then used to evaluate the mutants in order to determine the percentage of mutants that did not pass the tests.

Although mutation testing is powerful and has an active research state with a history of more than 40 years [2], it is a complex and computationally expensive testing approach that is seen primarily as a resource-intensive activity when compared to other testing methods. As a result, having support from automated tools is essential for carrying out mutation testing if one wants to develop it into a testing method that is both applicable in practice and usable across the industry.

This work provides a description of mutation testing as a method for measuring the validity of tests, as well as an analysis of eight existing state-of-the-art Java tools for the automation of mutation testing (MuJava, PIT, MAJOR, Judy, Jester, Jumble, Javalanche and Bacterio). Finally, based on the framework proposed by Amalfitano et al. [70], I selected two tools, namely PIT and MAJOR, to further comparison and evaluation through empirical experimentation. The experiment uses a set of real world open source projects from the Defects4J benchmark. A total of 740 tests suites were generated with the help of two state-of-the-art test generation tools: EvoSuite and Randoop. The performance of each mutation tool was evaluated and compared based on its capacity to reveal

faults, mutation score and execution time. In the end, the results demonstrate that, none of the tools was able to subsume the other.

Keywords: Mutation Testing, Software Testing, Mutation Analysis, Seeded Faults, JVM, Java, Equivalent Mutants, Mutation Operators, Cost Reducing Strategies.

Acknowledgements

First and foremost, I would like to express my gratitude to my Ph.D. Ana Paiva and Professor João Bispo, for their guidance, knowledge sharing, and wise counsel.

This journey would not be the same without the support of friends, MESW colleagues, and professors. Thanks to everyone who made this trip fun and interesting, and best of luck to everyone.

Finally, I want to thank my family for their support over the years, especially my wife for her encouragement, care, and constant presence, which has given me strength and understanding. And, of course, my son, who despite being 9 years old and in constant need of time and attention, kept saying, "Dad, go ahead!"

Luís Pinto

“Vision without action is merely a dream. Action without vision just passes the time. Vision with action can change the world.”

Joel Barker

Contents

1	Introduction	1
1.1	Problem Statement, Motivation and Goal	3
1.2	Relevance and Research	3
1.3	Structure of the Dissertation	4
2	Mutation Testing Background	5
2.1	Historical Background	5
2.1.1	The Mothra Project	5
2.1.2	Years of Research	6
2.2	Competent Programmer Hypothesis and the Coupling Effect	6
2.3	Mutation Analysis	6
2.4	Object-Oriented Challenges for Mutation Testing	8
2.5	Mutation Operators	11
2.5.1	Traditional Mutation Operators	11
2.5.2	Language Dependent Mutation Operators	11
2.6	Dealing with Equivalent Mutants	12
2.6.1	Techniques for equivalent mutants detection	13
2.6.2	Techniques to avoid equivalent mutants	13
2.6.3	Techniques for suggesting equivalent mutations	14
2.7	Cost Reducing Strategies	14
2.7.1	The Cost of Mutation testing	14
2.7.2	Selective Mutation	15
2.7.3	Incremental Analysis	16
2.7.4	Weak Mutation	16
2.7.5	Mutant Schemata Generation	16
2.7.6	Compiler-integrated Mutations	17
3	Relevant Work	18
3.1	Experimental Work on Mutation Testing	21
3.2	Java Mutation Testing Tools Comparison	23
3.3	Conclusion	25
4	Java Mutation Testing Tools	26
4.1	MuJava	26
4.2	PIT	26
4.3	Javalanche	27
4.4	Judy	28
4.5	Jumble	29

4.6	Jester	29
4.7	MAJOR	29
4.8	Bacterio	30
5	Design of the Experiment	31
5.1	Research Questions	31
5.2	Test Subjects	32
5.3	Test Suites	32
5.4	Mutation Testing Tools Selection	32
5.5	Analysis Procedure	34
5.6	Experiment Environment	35
6	Discussion	36
6.1	Experimental Results	36
6.2	Mutation Operators - A Complementary Analysis	41
6.3	Summary	44
7	Conclusion	45
7.1	Process and Results	46
7.2	Future Work	47
A	Tables	54
B	Charts	59

List of Figures

2.1	Generic Process of Mutation Analysis	7
6.1	Tools Fault Revealing Capability Matrix by Subject	37
6.2	Tools Fault Revealing Capability by Subject	38
6.3	Mutation Testing Execution Time Tools Comparison	39
6.4	PIT Fault Revealing Mutation Operators by Subject	42
6.5	PIT Fault Revealing Mutation Operators	42
6.6	MAJOR Fault Revealing Mutation Operators by Subject	43
6.7	MAJOR Fault Revealing Mutation Operators	43
B.1	Mutation Results Comparison: Chart Subject	59
B.2	Mutation Results Comparison: Compress Subject	60
B.3	Mutation Results Comparison: Lang Subject	60
B.4	Mutation Results Comparison: Time Subject	60

List of Tables

2.1	Class-level mutation operators	10
2.2	Mothra's "Key" Mutation Operators	16
4.1	Judy mutation operators	28
5.1	Subjects	32
6.1	Mutation Aggregated Results Comparison	40
A.1	Mutation Results Comparison: Chart Subject	55
A.2	Mutation Results Comparison: Compress Subject	56
A.3	Mutation Results Comparison: Lang Subject	57
A.4	Mutation Results Comparison: Time Subject	58

Chapter 1

Introduction

Software delivery is in the critical path to the success of businesses across all industries. Additionally, the level of complexity has never been higher. For engineering teams to succeed safeguarding their business, during annual downtime periods or big market swings, they must adopt the right practices in the right way. Effective and continuous software testing is one of those practices and probably the most important one.

Software Testing is the evaluation and verification that a software product or application performs as intended. Some of the advantages of testing include bug prevention, reduced development costs, and improved performance. There are numerous types of software tests, each with its own set of goals and strategies (i.e. Unit Testing, Acceptance Testing, etc.).

Software Testing has evolved over the years and has now become an integral part of software development, which began shortly after World War II ended. Testing first ensured that the software works as per the specified end-user requirements (Business Requirement or Functional Requirement) and was used to find the issues/bugs and also ensuring the proper functioning of the software. At that time, debugging was the primary testing approach, and it would remain that way for the subsequent twenty years. By the 1980s, development teams were shifting their focus away from identifying and addressing software defects and toward testing applications in real-world environments. It laid the groundwork for a more all-encompassing conception of testing, one that included quality assurance as an integral component of the software development life cycle. Then, testing was taken to an entirely new level by automation. With a variety of automated testing frameworks at hand, testers were enabled to do their work with greater efficiency and accuracy. Automation also improved the speed and accuracy of regression testing and sanity testing. With automated testing, in control of tests execution, more time can be saved on repetition tasks and this time can be spent on higher value tasks, such as exploratory tests. Continuous Testing is in the trend now and has become important with concepts like Continuous Integration and Continuous Deployment becoming popular. Then, to address the needs of continuous integration and delivery, we have turned to automated continuous testing. This is the essence of DevOps, a philosophy that

encourages collaboration between development and operations teams throughout the whole product life cycle. The idea is to expedite software delivery while reducing costs, improving quality, and managing risk. Using this testing method, teams do not need to wait until the program is built to begin testing. They can do tests considerably early in the development cycle to detect errors earlier, when they are easier to fix.

Testing software earlier promotes a more thorough testing process. Testing sooner in the software development life cycle helps not only maintain the software with higher quality but also reduces costs since detecting faults earlier winds up being not just easier but also cheaper to resolve.

But, **how do we know if our test suites are appropriate**, and how can the quality of the tests be improved so that we can test the most possible code behavior?

A particularly simple test adequacy metric is **code coverage**, which measures the proportion of code that are executed by a test suite [19]. It can help figure out how much of our source code is actually being tested and assess the quality of test suites. By executing a code coverage tool, we receive a code coverage report with the resultant percentage of code covered by our tests. However, there is no silver bullet for code coverage, and even a high percentage of coverage could be a problem if important parts of the application aren't being tested or if the tests already in place aren't strong enough to catch problems right away. That being said, **good coverage does not equal good tests** and, while, While reaching a high level of coverage is a worthwhile objective, it should be backed up by the creation of a thorough and reliable test suite that can confirm that neither the system as a whole nor individual classes have been compromised. So, **what other technique can be used to overcome this known code coverage weakness and help improve test suites adequacy?**

Mutation Testing is one such technique for testing evaluation. It helps understand what the tests are doing to the program being evaluated and makes it easier for test suites to find where faults are. By evaluating the quality of their test suites, testers and developers can understand how they can be improved. The basic principle underlying Mutation testing is to produce a large number of syntactically changed copies of the initial program, known as mutants, each having just one difference from the original program. The test suite then runs through all of the mutants to find out the percentage of mutants that failed the tests. Test suites are then measured by the percentage of mutants that they kill. As a result, new tests can be created, or existent ones can be changed, to kill additional mutants. The main goal is to assist testers in developing effective tests by finding bugs in test cases or parts of the code that don't get touched very often during execution.

The history of research on mutation testing goes back more than 40 years [2], and a significant amount of effort has already been put into the development of tools for the automation of mutation testing. Although, despite being around for several decades and thoroughly studied in research, mutation testing is still not widely adopted in industry [37, 68].

1.1 Problem Statement, Motivation and Goal

Mutation testing mandates the use of test cases that disclose the artificially introduced faults. Several empirical studies have already demonstrated the effectiveness of mutation testing in identifying these faults ([63]) and that it has, most certainly, the capacity to replace virtually all the structural testing techniques [33, 37]. It has been demonstrated that when test cases are able to distinguish the behavior of the original program from the defective one, they are also able to distinguish the expected behavior from the faulty one [63].

However, even though mutation testing is a powerful technique, it does have its limits. Applying mutation testing to real-world programs requires automated tools due to the vast number of faults involved. But, choosing the right mutation testing tool or tools to use for a specific use case is hard and challenging. According to Amalfitano et al., *"existing tools, even for the same programming language, differ substantially. For example, different tools implement different mutation operators, applied to source code or byte code, and produce different output artifacts, such as mutated source code and a mutant-test kill matrix. These differences and their corresponding trade-offs for different use cases are not always apparent, often due to a lack of documentation and/or empirical evidence. As a consequence, a researcher, educator, or practitioner may make suboptimal choices or may be forced to conduct a deeper investigation when choosing the most suitable tool for the use case at hand."* [70]. Although several studies exist in the literature, important characteristics of the tools that are fundamental to enabling users to make an informed decision have not been taken into consideration. Answering questions like "Is this tool actively maintained?" or "Is this tool compatible with Java 8?" is required by practitioners if they are in the process of selecting a tool to integrate mutation testing into a company's software development life cycle. In the same way, researchers will probably want to know about the different tools' capabilities, like what mutation operators are supported or if they can be filtered out, what cost-reducing strategies they support, etc. These are essential to help select the tools that support the intended studies.

The goal of this work is twofold. First, we present a comparison of the most important and relevant tools' characteristics to consider when in the process of selecting Java mutation testing tools. Second, we conducted a comparison experiment to show how a set of selected tools behave in terms of their efficiency and ability to find real faults.

1.2 Relevance and Research

Numerous research papers on mutation testing, its concepts, and tool comparisons have already been published. More than 100 research papers were analysed in order to assess their relevance to this work. Through the analysis, we found out that several studies providing an empirical evaluation of the Java mutation tools' effectiveness comparison were already done in the past [57, 64, 66, 65, 59]. These papers, however, directly focus on the specific aspects of the mutation testing tools under analysis and don't provide a comprehensive comparison of the tools' characteristics that led

to their selection for the study. In this work, we not only present a comparison of the tools under analysis but also follow a comprehensive framework for tool selection [70] which represents an important key differentiator from others.

1.3 Structure of the Dissertation

Chapter 2 provides an outline of the most significant concepts behind mutation testing. The process of creating mutants is presented, as are techniques for dealing with the two fundamental challenges of mutation testing: equivalent mutants and high computational costs.

Chapter 3 presents an historical overview of mutation testing and the most relevant research done in past decades, with special focus on Java mutation tools comparison studies.

Chapter 4 is about mutation testing in JVM environments. The new class-level mutation operators are presented, as well as key object-oriented concepts to keep in mind when working with Java programs. The following sections describe mutation testing tools for Java programs. The chapter concludes with a summary of existing experimental evaluations of the tools described.

Chapter 5 describes the design of the experiment and the followed methodology while Chapter 6 summarizes and evaluates the results. Finally chapter 7 present the conclusions of our study.

Some detailed data that supports our analysis and conclusions is provided in Appendix A. Tables (A) and Appendix B. Charts (B).

Chapter 2

Mutation Testing Background

This chapter provides a detailed overview of the mutation testing process and explains some terms associated with mutation testing that are essential to follow up on the subject.

2.1 Historical Background

Many subsequent research have contributed to the advancement of the mutation testing area since the publishing of the first study on mutation testing by DeMillo, Lipton, and Sayward with their paper titled "Hints on Test Data Selection: Help for the Practicing Programmer" [2].

Mutation testing relies on important principles such as the *Competent Programmer Hypothesis* and the closely related *Coupling Effect*. The initial research efforts, which took place in the latter half of the 1970s and the early 1980s, concentrated on program mutation at the unit level. Quite a few of the most essential theoretical topics were presented during this era. The investigation conducted by Budd uncovered not only the issue of equivalent mutants but also the concept of program neighborhood. Program neighborhood refers to the collection of programs that are extremely similar to the initial program and serves as the theoretical foundation for determining how to define mutation operators. It was also discovered by Budd that the complexity of the creation of mutants is equal to $O(Refs * Vars)$, being *Refs* the number of variable references and *Vars* the number of data objects. This was later validated by further research conducted in the 1990s by Offutt et al. [9].

2.1.1 The Mothra Project

The Mothra project was created at the Georgia Institute of Technology in the 1980s as a means to solve the many engineering problems faced with the application of mutation testing and became the first automated mutation system to be widely adopted. It was written in C targeting Fortran programs and implemented an intermediate language called MIT (Mothra Intermediate Language), to which the mutated programs were translated. This MIL includes several operators designed to support mutation. It aided in identifying the major barrier to the practical application of mutation

testing: mutation analysis is too slow and difficult for ordinary testers to utilize. As a result, researchers concentrated on cost-cutting measures such as do-fewer, do-smarter, and do-faster. Hundreds of institutions and labs have used Mothra in their own projects, leveraging its modularity and ease of change in order to apply improvements and adjustments to the tool, sharing those with other academics.

2.1.2 Years of Research

Following the Mothra project, new tools like muJava, Proteum, Csaw, Jumble, PlexTest, Heckle, and many others were developed. Object-oriented software (including the definition of object mutation operators), online applications, real-time and concurrent software have all begun to use mutation testing. New languages, like Java, Python, C, SQL, Ruby, PHP, and AspectJ, can now be used with mutation testing.

The current development shows that the community is striving to integrate mutation testing into the software development life cycle. To facilitate the use of mutation testing tools, new IDE plugins are being developed. For example, the Maven plugin for the PIT tool can be used during the build process, and the build only works if the mutation score reaches a certain level.

2.2 Competent Programmer Hypothesis and the Coupling Effect

According to the *Competent Programmer Hypothesis*, most programmers write nearly accurate programs[2]. This means that the tested programs work pretty well, and that the problems are just deviations from how the programs were supposed to work [62].

The *Coupling Effect* asserts that tests sensitive enough to discover all simple errors in a program are also sensitive enough to detect all complex errors [2]. A simple error is one that can be corrected with a single change to the source code, whereas a complex error is a mixture of simple errors. Thus, according to the Coupling Effect, complex errors are connected to simple errors [6].

Based on the above definitions, one can conclude that if the tests can recognize simple faults, they should also be able to recognize complex faults created from these simple faults. Mutation testing is a technique used to determine how well tests recognize simple faults in programs.

2.3 Mutation Analysis

Creating and killing mutants is at the core of the Mutation Analysis technique. Every single mutant is a modified copy of the original program that has been altered (mutated) by a single modification (mutation). These modifications are carried out in accordance with a set of predefined syntactic rules that are referred to as mutation operators. During the process of mutation testing, numerous modified versions of the original program, which are referred to as mutants, are created. After

that, tests are carried out on these mutants, and it is anticipated that the mutants will fail. When the test is unsuccessful, the mutant is considered to be killed, and there is no need to run any more experiments with this mutant going forward.

The process of applying the mutation testing starts by constructing the mutants test program [12, 7]. The detailed testing process is shown in Figure 2.1 [24].

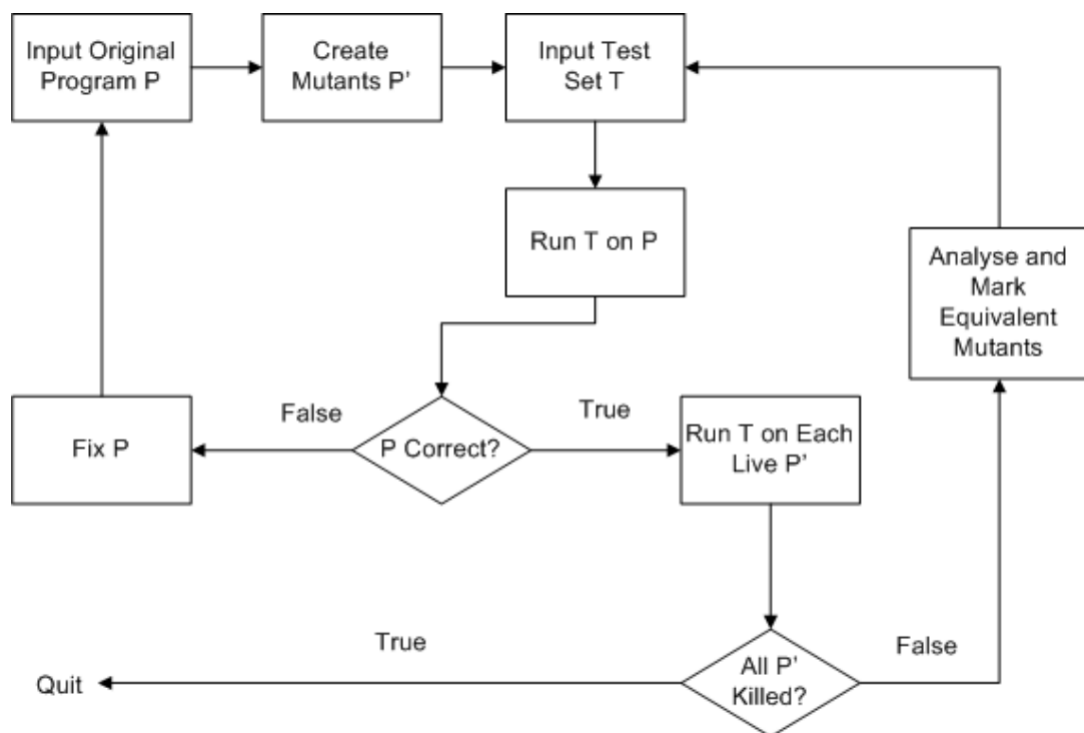


Figure 2.1: Generic Process of Mutation Analysis

In the next example, a mutation operator is used to change the original Java code in Algorithm 1 by replacing `==` with `!=` and generating the mutant in Algorithm 2. Other mutants can be generated by replacing `==` with `>` or with `<`.

Algorithm 1: Original Code

```

1  if(a == b) {
2      // do this thing
3  }
4  else {
5      // do this other thing
6  }

```

Algorithm 2: Mutated Code

```
1 if (a != b) {  
2     // do this thing  
3 }  
4 else {  
5     // do this other thing  
6 }
```

If none of the tests fail, the mutant is considered live. This could suggest one of two things: either the tests aren't sensitive enough to detect the code changes, or this mutant is equivalent. A syntactically equivalent mutant is different from the original, but it still means the same thing, so tests can't find them. Equivalent mutants are discussed in section 2.6 and are one of the most important challenges related to mutation testing.

The goal of mutation analysis is to determine the mutation score, which is the percentage of non-equivalent mutants killed by test data:

$$MS = M_K / M_T - M_E$$

- MS - Mutation Score
- M_K - Number of Killed Mutants
- M_T - Number of All Generated Mutants
- M_E - Number of Equivalent Mutants

If a test data set's mutation score is 100%, it is considered mutation-adequate. The test data is so much better the higher the mutation score is, which means more faults were discovered. This method results in iterative improvement of test data. Exploring live mutants can also aid in the detection and correction of problems that are concealed or occur during runtime.

2.4 Object-Oriented Challenges for Mutation Testing

While traditional modular languages like C work through function calls between the different modules, object-oriented languages consist of a set of interacting objects communicating through method calls. The usage of objects is supported by a set of specific object-oriented concepts like inheritance, polymorphism, and encapsulation. To be able to assess tests under this paradigm, mutation testing needs to understand how classes work in the context of the whole program and be able to generate mutants that can be used to test these concepts - this is known as inter-class mutation testing [41].

The mutation operators discussed thus far are used to change the content of methods and have no effect on object-oriented concepts. As a result, they are classified as method-level or statement operators. In response to object-oriented mutation testing, additional sorts of mutant operators, class-level operators, were invented. MuJava tool developers created the first class-level operators. The purpose of these class-level mutation operators is to change Java language features, including inheritance, polymorphism, access control, and type conversion. Table 2.1 [41] summarizes class-level mutation operators into four categories: Java-specific features; encapsulation; inheritance; and polymorphism.

Encapsulation – To ensure that classes, methods, access modifiers and fields are appropriately specified, a new mutation operator named AMC was developed. However, additional research concerning equivalent mutants generated by class-level operators [31] demonstrated that this operator generates a substantial number of incompatible or equivalent mutants. Consequently, this mutation operator is no longer employed.

Inheritance – An incorrect application of inheritance might result in a wide variety of faults. These forms of mutations are supported by mutation operators that seek to modify how inheritance is applied, such as method overriding, constructors definition and the utilization of the *super* or *this* keyword.

Polymorphism – This is a very strong object-oriented concept that provides the capability to refer to objects whose types are different from those that have been declared. The underlying type may be any of the type sub-classes that have been specified, or it may be an implementation of one of the declared interfaces. So, it's important to make sure that the tests cover the use of other kinds of bindings.

Group	Operator	Description
Encapsulation	AMC	Access Modifier Change
	EAM	Accessor method change
Java Specific Features	EMM	Modifier method change
	EOA	Reference/content assignment replacement
	EOC	Reference/content comparison replacement
	JDC	Java-supported default constructor creation
	JID	Member variable initialization deletion
	JSD	static modifier deletion
	JSI	static modifier insertion
	JTD	this keyword deletion
	JTI	this keyword insertion
Inheritance	IHD	Hiding variable deletion
	IHI	Hiding variable insertion
	IOD	Overriding method deletion
	IOP	Overriding method calling position change
	IOR	Overriding method rename
	IPC	Explicit call of a parent's constructor deletion
	ISD	super keyword deletion
	ISI	super keyword insertion
Polymorphism	OAC	Args of overloading method call change
	OMD	Overloading method deletion
	OMR	Overloading method contents replace
	PCC	Cast type change
	PCD	Type cast operator deletion
	PCI	Type cast operator insertion
	PMD	Member variable decl. with parent class type
	PNC	new method call with child class type
	PPD	Parameter variable decl. with parent class type
	PRV	Reference assignment with other variable

Table 2.1: Class-level mutation operators

2.5 Mutation Operators

"Mutation Operators are syntactic rules used to create mutants" [24]. The set of mutation operators is determined by the language of the program under test and the mutation tool used for mutation [14]. A mutation operator's objective is to create simple syntax changes based on common programming faults. Mutant operators have a crucial role on mutation testing. Consequently, many mutation operators have been proposed along the years and their impact explored by researchers [71, 30, 38, 50, 41, 42, 26, 14, 31, 24, 16, 28].

2.5.1 Traditional Mutation Operators

Based on syntactic criteria, mutation operators can be categorized into the following groups:

- **Statement Deletion** - These operators are used to delete single statements (e.g. continue, break, return in functions) or whole statements (e.g. while, for loop, if and switch statements, leaving only the body of those code blocks).
- **Statement Modification** - These kind of operators change the syntax of specific statements (e.g. replace switch cases, replace while with repeat-until, replace return values).
- **Boolean Expression Replacement** - These type of operators change boolean expressions (e.g. replace true with false, negate relational operators like == with != or <= with > and even replace whole conditional statements with true).
- **Variables and Constants Replacement** – Operators that change variable references/constants with the name of another variable/constant.

These mutation operators are known as traditional mutation operators. They are independent of any programming language, which means they can be defined in the majority of the languages.

2.5.2 Language Dependent Mutation Operators

There are additionally language-specific mutation operators used to modify language-specific constructs. The instances listed below are examples of those mutation operator groups:

- **Object-Oriented Operators** – These operators deal with JVM mutation testing specifics.
- **Concurrent Constructions Modification** – Used to introduce statement modifications in concurrent programs to support synchronization and resource management.
- **Aspect-Oriented Operators** – Used to modify constructs of aspect-oriented languages such as AspectJ.
- **Web Operators** – These apply modifications and tag replacements on HTML or JSP pages in web applications.

2.6 Dealing with Equivalent Mutants

Equivalent mutants are copies of the original program that differ syntactically but are otherwise identical to the original in terms of their semantics. For example, the Algorithm 3 computes the maximum from an array of integers. By mutating its conditional operator from `>` to `>=` we get a syntactically copy (Algorithm 4) of the original program but the final result is exactly the same. Another example of an equivalent mutant is in Algorithm 5. In this case the counter in the initial loop is set to 0 instead of 1. The final result of the function, however, is not impacted.

Original Code

```
1 void max(int[] arr) {
2     r = arr[0];
3     for (i = 1; i < arr.length; i++) {
4         do {
5             if (arr[i] > r) {
6                 r = i;
7             }
8         }
9     }
10 }
```

Equivalent Mutant 1

```
1 void max(int[] arr) {
2     r = arr[0];
3     for (i = 0; i < arr.length; i++) {
4         do {
5             if (arr[i] > r) {
6                 r = i;
7             }
8         }
9     }
10 }
```

Equivalent Mutant 2

```
1 void max(int[] arr) {
2     r = arr[0];
3     for (i = 1; i < arr.length; i++) {
4         do {
5             if (arr[i] >= r) {
6                 r = i;
7             }
8         }
9     }
10 }
```

Equivalent mutants [60, 58] are a serious problem for mutation testing because they distort the mutation score. As already defined, mutation score is the percentage of non-equivalent mutants killed by test data. Finding equivalent mutants consumes a lot of time (according to Schuler and Zeller approximately 15 minutes per equivalent mutant [52]) and there is still no automated algorithm to detect all of the equivalent mutants [39].

According to Madeyski et al., the rate of equivalent mutants could range between 4% and 39%. A similar conclusion was demonstrated by Grün et al. [34] with up to 40% live mutants seeming to be equivalent while mutating various classes. This can result in programmers wasting their time writing more accurate tests and trying to find bugs where there are none. Specially, when manually checking mutant equivalence can be error-prone [3]. As a consequence, analysis done on big real world software projects often ignores equivalent mutants and only computes the Mutation Score Indicator:

$$M_{SI} = M_K/M_T$$

Next, several methods that try to solve the problem of equivalent mutants will be introduced. The classification presented here is based on the research of the author of Judy mutation testing tool L. Madeyski et al. [39]. His research on equivalent mutation, which he did up until 2014, led to the development of three main ways to get around the problem of equivalent mutation.

2.6.1 Techniques for equivalent mutants detection

Compiler optimization techniques – This strategy is based on the premise that most equivalent mutants are either optimizations or de-optimizations of the original program. When a mutant fulfills any of the specified code optimization conditions, the algorithm can determine that the mutant is equivalent to the original [13].

Mathematical constraints – A constraint in mutation testing is the circumstance under which a mutant will get killed. Constraints are used to generate test data automatically. If a mutant gets killed by a test case, then it is said that the test case satisfies the constraint system. The mutant is equivalent if the constraint system cannot be true, meaning there is no test case that can kill the mutant [15].

2.6.2 Techniques to avoid equivalent mutants

Selective Mutation – Use only those mutation operators that are capable of producing statistically less equivalent mutants. This technique is also one of the performance optimization techniques that will be discussed later in greater detail.

Higher Order Mutation Testing – Higher order mutants (HOMs) is a technique that creates mutants with more than one mutation. To accomplish this, the Judy tool creators introduced four different algorithms, namely Last2First, DifferentOperators, ClosePair and RandomMix. The idea was to conduct empirical experiments by combining single-order mutants (HOMs), or mutants with just one mutation, with second-order mutants (SOMs), or mutants with two mutations. Results were very promising, revealed a reduction of 85.65% and 87.77% of equivalent mutants, and, at the same time, the number of generated mutants was also reduced by 50% [39].

2.6.3 Techniques for suggesting equivalent mutations

Examining the impact of equivalent mutants on coverage – Created by the developers of the Javalanche mutation testing tool, this technique is based on measuring program control flow and trying to detect if a mutant somehow changes the execution. This is performed by computing code coverage on the original program, in each mutant, and finally comparing the coverage disparity. The greater the coverage disparity, the less likely the mutant is equivalent. [20].

Using the impact of dynamic invariants – Also developed by the creators of the Javalanche tool, this other technique has its roots in the assumption that all non-equivalent mutants must modify the program's behavior; therefore, dynamic invariants are tested for every function. Invariants for the original program are first created based on variable and field accesses in a function, and then conditions are checked during mutant execution. If the condition isn't met, there is an invariant violation, and less likely is the mutant equivalent. [35].

According to Madyeski [51], the vast majority of research is devoted to avoiding and suggesting techniques given the huge computing cost and complexity of implementing techniques to detect equivalent mutations. However, detection techniques give no false positives compared to suggesting techniques. In the sections that follow, we describe each category's most relevant techniques.

2.7 Cost Reducing Strategies

Mutation testing is a powerful technique but it also has great computational costs. Despite its already relatively long history of research, the fact is that these great computational costs have kept mutation testing from being a commonly used practice for many years.

2.7.1 The Cost of Mutation testing

Budd's research demonstrated that the number of mutations generated is related to the number of variable references multiplied by the number of data objects [5]. Following that, each mutant must be run at least once, maybe multiple times, resulting in a significant computational cost. In general, the following are the most costly steps of mutation testing that must be optimized:

1. **Mutant Generation** – Besides the significant computational costs, memory utilization is also substantial during this phase. Mutation operators must be run on the original code, and the resulting mutants must be saved for later use.
2. **Mutant Compilation** – In this phase, the generated mutants must be compiled, which can result in very high computational costs, especially with bigger programs.
3. **Execution of tests** – The purpose of mutation testing is to eliminate all mutations. This means that tests need to be executed for each and every mutant to try to kill them. The bad here is that even live mutants are tested, resulting in high computational costs.

To tackle mutation testing high computational costs and open the door for its use in the software development life cycle, various cost reducing strategies were researched and developed. These are divided into three main categories:

1. **Do-fewer strategies** – Reducing the number of generated mutants is one technique to reduce the computational cost [16]. Selective Mutation is one of these strategies by applying only the most critical mutation operators. Mutant Sampling is another which applies only a randomly selected mutant operators subset in a single test run.
2. **Do-smarter strategies** – This is a technique, based on parallelization of work, that consists of avoiding having to execute all tests to completion based on the assumption that most probably they are unnecessary. These strategies use parallelization to avoid the complete execution of all tests when it is most probably not needed [10]. Incremental Analysis and Weak Mutation strategies are techniques that fit into this category.
3. **Do-faster strategies** – The idea behind this strategy is to optimize and speed up the generation and compilation phases. Mutant Schemata Generation, Byte-code mutation, Separate Compilation and Compiler-integrated Mutation are examples of such strategies belonging to this category [11].

2.7.2 Selective Mutation

The purpose of selective mutation is to attain maximum coverage while producing the fewest number of mutants. Because mutants are generated by using mutation operators, the ones that produce the most mutants that are readily destroyed are excluded. According to Offutt et al. "Complexity reduction of mutants generation is from quadratic ($O(Refs * Vars)$) to linear ($O(Refs)$)" [16], and at the same time maximize efficacy of non-selective mutation. One of the first studies [16] was done with mutation operators from Mothra tool and consisted on gradually excluding two, four and six of the most populous mutation operators.. The results proved the hypothesis by executing test cases that killed almost 100% of mutants. Consecutive studies demonstrated that only 5 of the 22 mutation operators used by Mothra are "triggering" operators, providing about the same coverage as non-selective mutation [9]. Table 2.2 describes those 5 mutation operators.

Operator	Description
ABS (Absolute Value Insertion)	Replaces each arithmetic value with 0
AOR (Arithmetic Operator Replacement)	Replaces each arithmetic operator with every syntactically legal operator
LCR (Logical Connector Replacement)	Replaces each logical connector (AND, OR) with several kinds of logical connectors
ROR (Relational Operator Replacement)	Replaces one relational operator with other relational operator
UOI (Unary Operator Insertion)	Inserts unary expressions in front of expressions

Table 2.2: Mothra's "Key" Mutation Operators

2.7.3 Incremental Analysis

Incremental Analysis aims speeding up mutation testing during the software development life cycle. It is used by storing the results from previous runs allowing to test only the code affected by the last change.

2.7.4 Weak Mutation

Approaches to mutation testing are considered strong mutation. Mainly for larger programs, waiting for each mutant to end execution can be very expensive. Weak mutation tries to overcome this by using an approximation like technique suggested by Howden [61, 8]. Rather than comparing the final state of processed mutants, the inner state is compared to the initial state as soon as the altered part of code has been carried out. The study from Offutt and Lee compares different strategies for weak mutation, namely *Expression Weak mutation (EX-WEAK/1)*, *Statement Weak Mutation (ST-WEAK/1)* and *Basic block Weak Mutation (BB-WEAK/1|N)*. With their experiment, the authors shown that ST-WEAK/1 and BB-WEAK/1 produced results almost as good as the ones produced by strong mutation taking, at least, minus 50% of execution time [16].

2.7.5 Mutant Schemata Generation

Automated mutation testing systems based on the traditional interpretative method are time-consuming and labor-intensive to develop [16]. Offutt introduced a new execution model named Mutant Schemata Generation (MSG) that, instead of creating and compiling each mutant, it encodes all mutations into one source-level program – a “metamutant” [46]. The metamutant is compiled once and then different mutations are applied by passing a run number. Benchmarks showed that using MSG significantly improves mutant generation.

Original Code

```
1 a = b + c ;
```

Metamutant

```
1 switch (#Run) {  
2     case 1:  
3         a = b - c;  
4     case 2:  
5         a = -b + c;  
6     case 3:  
7         a += b + c;  
8     case 4:  
9         // ...
```

2.7.6 Compiler-integrated Mutations

With this method, the original program is compiled by a special compiler capable of detecting the effects of mutations and preparing code patches. These adjustments are subsequently applied during the code execution [16], resulting in faster execution of the mutants due to compile optimizations. The most difficult thing to achieve is creating such a specialized compiler [11].

Chapter 3

Relevant Work

In 1971, the student Richard Lipton suggested the concept of mutation testing for the first time [24]. It was after developed and published in the paper “Hints on Test Data Selection: Help for the Practicing Programmer” by DeMillo et al. [2] and Hamlet [1] in "Testing programs with the aid of a compiler". In 1980, Timothy Budd built the first mutation testing tool as part of his Yale’s Ph.d. thesis, titled *"Mutation Analysis of Program Test Data"* [4]. Since then, mutation testing was increasingly and widely studied, enhancing the field with various kinds of techniques and attempting to make mutation testing more applicable to real-world situations. Mutation testing is not only useful at the software implementation level. It has also been applied to Finite State Machines, Stelle Specifications, State charts, Network protocols, Petri Nets, Web Services and Security Policies [37].

Actually, research in mutation testing has a vast and rich history [24]. According to Offutt et al., "Research in mutation testing can be classified into four types of activities [16]:

1 - Defining mutation operators

In this line of investigation, new types of testing and mutation operators for different languages are defined. It is essential to have a solid collection of mutation operators in order for mutation testing to be successful, and different operators are required for each language, language specifics, runtime platform, and types of testing.

In the paper *"The Class-Level Mutants of MuJava"* [31], Offutt, Ma, and Kwon introduce enhancements to the Java mutation testing tool MuJava with a number of brand-new class-level mutation operators, improvements to existing mutation operators, and an equivalent detection mutant engine. Then, they did an experiment to look at the traits of class-level mutants made from 866 classes belonging to six open-source Java projects. As a result, the authors "found that more than 70% of the class-level mutants were equivalent, far more than the 5% to 15% found with unit-level mutants", showing that there are an average of 46 per class fewer class-level mutants than unit-level mutants. They conclude by stating that mutation testing is much more practical when applied at the inter-class testing level although often seen as being too costly for practical unit testing.

To Ma, Kwon, and Offutt, "The effectiveness of mutation testing depends heavily on the types of faults that the mutation operators are designed to represent. Therefore, the quality of the mutation operators is key to mutation testing". In their paper *"Inter-Class Mutation Operators for Java"* [26], the authors observe that Object-oriented (OO) programs have a tendency to be structured differently compared to conventional programs, with OO languages posing new challenges due to their specific characteristics such as encapsulation, inheritance, and polymorphism. According to Offutt et al. [25], these specificities have the potential to introduce new types of faults. Hence, mutation operators designed for procedural programming languages are insufficient; instead, new language operators created specifically for object-oriented programming are necessary. The authors also state that, even though there is already research about developing mutation operators for OO software [21, 22, 16], they consider these previous attempts unsuccessful by lacking a general fault model available and the OO mutation operators are incapable of not only handling all fault types as well as not handling all Java language features. They then proceed to offer a collection of class mutation operators for Java with the intention of overcoming the weaknesses of earlier Java operators. These operators include improvements to already existing mutation operators in addition to a number of new ones. The goal was to enhance the reliability of OO software through the development of mutation operators integrating characteristics of Java to facilitate inter class-level testing and assist testers in finding faults by using object-oriented language specific features like access modifiers, polymorphism, inheritance and overloading.

Most of the tools and the correspondent mutation operators are mainly targeted at web and desktop applications. Paiva et al., aware of this, specifically targets mobile applications in their paper *Testing When Mobile Apps Go to Background and Come Back to Foreground* [67]. The authors present three new mutation operators specifically created to help detect "failures related to the non-preservation of users' transient UI state when mobile applications go to background and then come back to foreground" [67]. They explain the whys that motivated them to create these new mutation operators and conducted an illustrative case study of the approach by evaluating a mobile testing tool called iMPAcT. The study focuses on 56 real-world apps from the Google Play Store, although others may be used instead, covering different categories. The apps were selected from a bigger set according to a predefined criteria, also present in the study. In total, 611 mutants were generated and used to evaluate the quality of the test cases generated by the iMPAcT tool. However, according to the authors, any other mobile testing tool could be used. The authors then showed the results of the experiment, talked about where the results came from, and made suggestions for how to improve the iMPAcT tool so that it could kill more mutants and get better results.

Software testing has been, since forever, a challenging activity (Bertolino 2007) and its increasing complexity makes testing the entire software behavior infeasible (Moreira et al. 2017). According to Paiva, Restivo, and Almeida, "software test automation may be a way to deal with time restrictions and complexity" [69], so in the paper *"Test case generation based on mutations over user execution traces"*, the authors came up with an approach where user execution traces are used as

the source for test cases generation. They capture the user interactions with the web application and then the execution traces are analyzed to extract filtered information needed to build an initial set of test cases and subsequently generate new test cases. The objective is to use a set of mutation operators on the first set of test cases to make new ones that test different parts of the application under test. The authors defined and described five mutation operators (remove step, repeat step, swap step, insert empty, and add back) that when applied can be used to generate candidate test cases and automatically discard them if similar to the existing ones. To validate the approach, the authors executed two case studies and concluded that "the Levenshtein distance has impact over the subset of candidate test cases that are considered relevant. A distance higher than 1 reduces by 10% the percentage of accepted test cases" [69]. The authors also propose to perform more experiments in the future to further analyze the quality of the final set of test cases and to extend the set of mutation operators.

2 - Developing mutation tools

Over the last decades, several mutation tools have been brought to life by researchers and software engineers, targeting different languages and platforms in order to support the mutation testing practice. This activity is covered in Chapter 4 with a special emphasis on Java mutation testing tools.

3 - Reducing the cost of mutation analysis

Creating and killing mutants is at the core of the Mutation Analysis technique. Every single mutant is a modified copy of the original program that has been altered (mutated) by a single modification (mutation). Within the process of mutation testing, a large number of mutants are created. After that, tests are carried out on all these mutants. This process results in high computational costs, which become the principal drawback of mutation testing. Trying to overcome this, many researchers have tried over the years to come up with strategies that reduce the cost of mutation analysis. Offutt et al. confirm it in the paper *"An experimental determination of sufficient mutant operators"*, where they state that *"The principal expense of mutation is that many variants of the test program, called mutants, must be repeatedly executed"*. They assess the cost of mutation in terms of the number of mutants produced before proposing and evaluating a technique capable of reducing the number of mutants by an order of magnitude. The idea was that by applying selective mutation, which lowers the number of mutants, the cost of mutation testing would also be lower. The authors then compared selective mutation testing to normal (or non-selective) mutation testing and quantified the savings obtained. Their results back up the idea that selective mutation testing is almost as good as normal mutation testing. It gets almost as much coverage as non-selective mutation, but with significantly fewer mutants.

Other studies were already mentioned in Section 2.7, where several cost-reducing strategies are discussed.

4 - Experimentation with mutations

This will be the subject of the next section (3.1). There we summarize the research related to this specific activity as the body of knowledge to which this work adds.

3.1 Experimental Work on Mutation Testing

To date, several researchers have already investigated the effectiveness or efficacy of mutation testing. Andrews et al. [29] selected eight well-known C programs in order to make a comparison between defects that were hand-seeded and those that were generated by automated mutation engines. The authors came to the conclusion that faults generated through automated mutant generation were more accurate representations of the problems that occur in the actual world. On the other hand, faults that were inserted by hand undervalued the effectiveness of test suites because they ended up emulating faults that would most likely never occur in the real world. The authors also discovered that it was more difficult to spot errors that had been planted by seasoned developers.

Some researchers have broadened the application of mutation testing to include specification analysis - instead of changing a program's source code, inputs and outputs of a certain executable unit get altered through specification-based mutation analysis. Murnane and Reed [23] demonstrate that the usefulness of mutation analysis must be validated against more typical black box techniques that use this technique, such as boundary value and equivalence class splitting. By using equivalence class and boundary value analysis testing approaches, the authors created test suites for a data-vetting and a statistical analysis program. The test cases generated by these techniques were then compared to the test cases obtained by mutation analysis to identify redundant tests and assess the utility of any additional tests generated. The case study found that the test cases provided by standard specification analysis techniques and those generated by mutation analysis were only 14-18% equivalent. This finding suggests that using mutation analysis will discover many relevant test cases that standard specification techniques will not.

Analysis testing was contrasted with all-uses testing by Frankl and Weiss [17] using a collection of popular C programs that included faults that occurred as a result of natural occurrences. In all-uses testing, a test suite is generated to induce and anticipate outcomes from every possible path across a given system's call graph. The authors acknowledge that an all-uses test suite does not yet exist for several of the programs that comprised their sample population. In five out of the nine case studies, mutation analysis was shown to find more of the known faults than did all-uses testing. As the authors show, there were many times when their tests killed every mutant but didn't find the natural fault, even though they had a high mutation score.

Just, Schweiggert, and Kapfhammer in their paper *"Using Conditional Mutation to Increase the Efficiency of Mutation Analysis"* [43] presented a versatile approach, based on transforming the

abstract syntax tree (AST), called *conditional mutation* to increase the efficiency of mutation analysis, reducing significantly the time overhead required for generating and executing the mutants. Results are reported for eight investigated programs up to 373,000 lines of code and 406,000 generated mutants. According to the authors, "the approach is versatile, programming language independent, and can be integrated within the compiler", enhancing it to easily run on commodity workstations since the overhead that induces is negligible. The total overhead reported for compiling 406,000 mutants for the largest investigated program was only 33% when compared with the default compile time. The authors also claim that the average execution time required for the respective test suites is just 15% longer than the normal testing time, which is orders of magnitude faster than compiling each mutant separately.

Also, from Just, Schweiggert, and Kapfhammer, the paper "*MAJOR: An Efficient and Extensible Tool for Mutation Analysis in a Java Compiler*" [44] presents MAJOR - a fault seeding and mutation analysis tool capable of facing some of the challenges of mutation analysis. To the authors, MAJOR is able to reduce the mutant generation time and enable efficient mutation analysis, referring that the tool "*has already been successfully applied to large applications with up to 373,000 lines of code and 406,000 mutants*". They also consider MAJOR extensible (through its domain specific language), ease to use and efficiency, making it an ideal platform for the study and application of mutation analysis.

From Just, Kapfhammer, and Schweiggert, the paper "*Do Redundant Mutants Affect the Effectiveness and Efficiency of Mutation Analysis?*" [48], expands prior research that found that a subset of all available mutation operators is adequate to achieve significant results, and examines these operators at a finer level, demonstrating that their original specification implies redundancy in the final collection of mutants. The paper includes a case study that demonstrates how prevalent these redundancies are in real-world applications and how the incorporation of redundant mutants results in an inaccurate mutation score. The study demonstrates that the removal of redundant mutants reduces the runtime of mutation analysis by up to 34% and that redundant mutants can lead to a mutation score that is misleadingly overestimated by as much as 10%. Overall, the paper shows that it is possible to make a mutation analysis system more effective and efficient by finding and getting rid of mutants that are already there. Another paper from Just, Kapfhammer, and Schweiggert, titled "*Using Non-Redundant Mutation Operators and Test Suite Prioritization to Achieve Efficient and Scalable Mutation Analysis*" [47], investigates the reduction in generated mutants by replacing conditional (COR) and relational (ROR) operators with a limited but sufficient number of mutants. They examined ten real-world applications, each with 400,000 lines of code and a total of more than 550,000 mutants generated. The analysis found a reduction in the number of generated mutants up to 37% and on average by more than 25%. The authors, however, stress out that using non-redundant mutation operators in isolation does not guarantee that the mutation analysis will be efficient and scalable. To confirm the case, the authors conducted an experiment based on the same ten open-source applications and observed that by using information about the

runtime and test mutation coverage for prioritization and combining it with non-redundant operators, the total cost of mutation analysis can be reduced by up to 65%. This was proved to be true by evaluating an efficient workflow that takes advantage of the redundancies and runtime differences of test cases to reorganize and partition the related test suite.

Scalability problems make it hard to do mutation analysis since a test suite has to be run for each of a large number of mutants. Although numerous optimizations proposed by previous research exists, this scalability issue persists and is one of the reasons why mutation analysis is rarely used in practice. In the paper "Efficient Mutation Analysis by Propagating and Partitioning Infected Execution States," Just, Ernst, and Fraser [54] uses information that is only available at runtime to find ways to cut down on the number of executions even more. The authors were able to successfully apply the planned enhancements to the Major mutation framework and empirically evaluate those improvements on 14 open source applications, which resulted in a reduction of the amount of time necessary to complete the mutation analysis by an average of 40%.

A test suite is only good if it detects real faults, and mutation testing's goal is to assess the adequacy of test suites. According to Just et al. *"there is minimal empirical proof to support the use of 36 mutants as a replacement for real faults"*, so, in their work *"Are Mutants a Valid Substitute for Real Faults in Software Testing?"*,

According to Just et al. there is little experimental evidence to support the use of mutants as a replacement for real faults, so, in their paper *"Are Mutants a Valid Substitute for Real Faults in Software Testing?"*, the authors investigate if mutants are a valid substitute for real faults, that is, whether the capacity of a test suite to discover mutants is connected with its ability to find real flaws that developers have addressed. They then applied mutation analysis to a set of 357 real faults within five open source applications, comprising a total of 321,000 lines of code, with 480 test suite pairs consisting of developer-written tests, and 35,141 automatically-generated test suites that may or may not find real faults. The results show that there is a statistically significant link between finding mutations and finding real bugs, regardless of how much code is covered. With controlled code coverage, 73% of real faults are connected to mutants created by the most common mutation operators used, but the number of mutants connected to a single real fault is minimal. The authors concluded by showing some of mutation analysis' limitations and giving concrete suggestions about how mutation analysis could be improved.

3.2 Java Mutation Testing Tools Comparison

Mutation tools attempt to automate and accelerate the development of fault-injected versions, known as mutants, as well as the execution of test suites on those mutants. In the following papers, different Java mutation tools for different kinds of use are analyzed at and compared.

In 2010, a research study, titled *"Judy – A Mutation Testing Tool for Java"* [39], was conducted in 2010 by the makers of the Judy tool to present its key features along with some theoretical

description of other mutation testing tools like MuJava, Jester, and Jumble. The authors also conducted an experiment, using a set of Apache Commons libraries, where the performance of Judy and MuJava is compared regarding the number of mutants produced per second (NMPS). The authors showed that Judy successfully produced, per second, at least ten times more mutants when compared to MuJava.

The usability of mutation testing was under examination in the research *"Is Mutation Testing Scalable for Real-World Software Projects?"* [45] from 2011. The experiment, executed with Eclipse, focused on the performance of generating mutants using MuJava, Jumble, and Javalanche with unsatisfying results. MuJava was the only tool capable of creating mutants, producing 123 class-level mutants and 30947 method-level mutations in around 6 hours. Jumble and Javalanche failed due to configuration and low performance. With this experiment, the authors proved that mutation testing did not have enough performance to be used in real-world software projects.

In 2013, Delahaye and Du Bousquet, in a study titled "A Comparison of Mutation Analysis Tools for Java" [49], explains mutation testing fundamentals and compares Javalanche, Judy, Jumble, MAJOR, and PITest. The study also includes some theoretical information regarding these tools as well as a comparison of some selected theoretical metrics. The results showed that Jumble, MAJOR, and PITest were able to complete the analysis for each project, whereas Judy generated the most mutations and Javalanche generated the fewest. In conclusion, although PITEST generates a large number of very easy-to-kill mutants, it reveals itself as the most framework-compatible tool, while Judy and MAJOR present themselves as the most natural successors to MuJava for research. Therefore, the research reveals that mutation testing tools require significant enhancements before they can be utilized in real-world applications.

The research "Experimental Comparison of Automated Mutation Testing Tools for Java" [59] selected for comparison MuJava, Judy, Jumble, Jester, and PITest. The experiments consisted of running a set of short programs (17-111 LOC) and demonstrated that PITest generated the fewest number of mutants compared to the other tools, whereas MuJava and Judy generated a considerable higher number of mutants than the other tools. Except for PITest, which produced a 25% higher average mutation score than the other tools, all tools produced nearly identical average mutation scores. Based on what the research found, it is suggested that a new mutation system, capable of faster mutants creation and execution, be made for Java.

The last research is "How do Java mutation tools differ?" from Amalfitano et al. The main goal of the work is to provide a comprehensive mutation tool comparison framework encompassing five dimensions (Tool version, Deployment, Mutation process, User-centric feature and Mutation operators), each with multiple attributes. The paper then uses the proposed framework to highlight the similarities and differences of eight state-of-the-art Java mutation tools (MuJava, PIT, MAJOR, Javalanche, Jester, Judy, Judy and Bacterio). The proposed framework can be used to identify the most suitable mutation tool for a given use case, whether it be Research, Education, or Practice.

Lastly, the study informs future work about the benefits of adopting a set of practices crucial to support a better informed decision regarding mutation testing tool selection, such, standardized mutation operator descriptions.

3.3 Conclusion

Several studies and experiments comparing Java mutation tools have already been conducted over the years by researchers. However, none of those studies focuses on the effectiveness of the tools considering their fault revelation capability, execution time, and test suite adequacy assessment using real-world Java projects with automated test suites generation. While those experiments use specific features of the tools to focus on certain aspects of mutation analysis, requiring deep knowledge of their inner workings, this experiment uses the tools' default settings and aims to give researchers, students, and practitioners an idea of how well these tools behave, and what they can do right out of the box. A comprehensive comparison of the tools that helped us select the ones we used in our experiment is also given.

Chapter 4

Java Mutation Testing Tools

While the previous chapters gave an overview of mutation testing, its main concepts, approaches and existing relevant research, this chapter focuses on describing existing Java mutation testing tools as well as the results of experimental comparisons of these tools done by other researchers. Tools that are experimental or difficult to obtain were discarded. The selected tools are: MuJava, PIT, Javalanche, Judy, Jumble, Jester, Bacterio and MAJOR. A comparison of these tools will be presented in this section, as well as their most interesting features.

A more detailed view of the tools characteristics can be found on the work done by Amalfitano et al. [70]).

4.1 MuJava

MuJava is one of the first Java-based mutation testing tools and considered by many the testing platform for mutation testing research. It was developed by Offut et al.[18] in 2004 as a research project and released under the open-source Apache License 2.0. It's not under active development and its last updated is from August 2015. The Java latest supported version is Java 7 while JUnit4 latest supported unit tests framework. MuJava can be used through its standalone GUI application or through an Eclipse plugin called MuClipse. It supports both method-level operators as well as several class-level operators. MuJava uses selective mutation strategy and uses the same five selective method operators from the Mothra project. The tool allows more than one class to be selected, but only one test is allowed to be executed at a time. There are three strategies for mutant creation: code mutation, byte-code mutation or MSG strategy. There is no equivalent mutants strategy implemented and MuJava output provides only the number of generated mutants, time of run and the mutation score.

4.2 PIT

PIT, in comparison to MuJava, is a much younger mutation testing tool. According to pitest.org, PIT is a "*state of the art mutation testing system, providing gold standard test coverage for Java*

and the jvm. It's fast, scalable and integrates with modern test and build tooling" and intended to be used in real world mutation testing. PITest is released under the open-source Apache License 2.0, is still under very active development. At the time of writing, PIT's version was 1.9.3 released on July 2022. PITest is compatible with Java 5+ and supports both junit4 and TestNG6 as unit testing frameworks. With its extensible plugin system, PITest enjoys the existence of several 3rd party plugins, not only for the Eclipse and IntelliJ IDEs but also for Maven and Gradle build automation tools. Although operators can be selected, classes and tests can only be specified by defining the packages to use. The tool creates mutants automatically, but they are not available for selection or preview. PIT supports just 13 method-level mutation operators (Conditional Boundary, Negate Conditionals, Math, Increments, Invert Negatives, Return Values, Void Method Call, Constructor Calls, Inline Constant, Non Void Method Call, Remove Conditionals), 7 of them active by default. This also includes 2 experimental operators (Member Variable, Switch).

PIT employs cost-cutting measures such as byte-code mutation, incremental analysis, and parallel execution. The mutants are generated based on code coverage, which means that only the methods tested are mutated. PIT facilitates the avoidance of equivalent mutants by picking default mutant operators that do not produce a large number of equivalent mutants. In the testing summary, there is information about how long it took to figure out the code coverage, how many mutants were made and how many tests were run on them, how many tests were run in total, the overall mutation score, and the mutation score for each mutant operator.

PIT offers 3 types of output files – HTML, CSV and XML providing a close overview of which mutants were created and which tests managed to killed them.

4.3 Javalanche

Javalanche is a research project of D.Schuler et.al. and was released under open-source LGPL v.3. Java 6 is the latest supported Java version, having JUnit4 as the unit testing framework. The tool also supports integration into Eclipse through a plugin. The mutants are automatically generated according to line coverage, although it is possible to select specific classes and tests [36]. Javalanche offers 7 method-level operators (Replace Constant, Negate Jump, Arithmetic Replace, Remove Call, Replace Variable, Absolute Value, Unary Operators) and 2 concurrent operators (Replace Thread Call, Monitor Remove). None of these operators can be selected. Javalanche is, however, no longer under active development.

Javalanche uses several cost reducing strategies like: selective mutation, byte-code mutation, parallel execution and MSG. Javalanche implements the equivalent mutations suggestion technique described before in Subsection 2.6.3. Some simple statistics, including the percentage of equivalent mutants, are the only output the tool provides.

4.4 Judy

Judy is a research project of L.Madeyski et al. from Wroclaw University of Technology. It is released under BSD License and implements FAMTA Light [39] mutation testing approach based on pointcuts which are special points in the program flow. The latest version is Judy 3 from January 2019 and is still under active development. Judy 3 supports only Java 8 and JUnit4 framework. Judy offers only a simple command-line working environment. Mutants are automatically generated, although they are not available to be selected, and it is possible to provide the path to classes and tests as well as define the mutation operators. The mutation operators available are not only the same set of mutation operators from MuJava (both method and class-level) but also many others with total number of 37 mutation operators (56 including special types of method-level operators) which are all active by default.

Operator	Description	Operator	Description
AIR	Replace basic binary operator (method)	EOA	Replace reference assignment
JIR	Replace jump instructions (method)	EOC	Replace reference comparison
LIR	Replace binary logical instructions (method)	JDC	Delete the implemented default constructor
SIR	Replace shift instructions (method)	JID	Delete field initialization
IOD	Delete overriding method	JTD	Delete this keyword
IOP	Relocate calls to overridden method	JTI	Insert this keyword
IOR	Rename overridden method	CCE	Clears the collection
IPC	Delete super constructor	REV	Reverses the list
ISD	Delete super keyword	LSF	Shuffles the list
ISI	Insert super keyword	DUL	Duplicates element in the list
OAC	Change order or number of arguments	ORV	Reverses the order of collections with ordering
OMD	Delete overloading method declarations	CST	Changes LinkedHashSet to HashSet
OMR	Change overloading method	FBD	Break statements removal from switch
PLD	Change local variable type to super class	SCR	Stream closing removal
PNC	Call new with child class type	CCD	CompareTo method distortion
PPD	Change parameter type to super class	EGE	Generalization of caught errors
PRV	Change operands of reference assignment	CSR	Collection synchronization removal
EAM	Change an accessor method name	CLR	Change list to set
EMM	Change a modifier method name		

Table 4.1: Judy mutation operators

By default, Judy uses byte-code mutation as well as parallel execution. Judy 3 initial release removed default support for HOM which will be released and backported as separate plugins for the different HOM strategies in the future. This change invalidates most of Judy previous research based on Judy 2.

Judy creates a summary report after each run containing the execution duration on mutants generation, test runs, overall runs, mutation score, and the used test cases. An XML report is also created, which requires some kind of GUI tool to be easily interpreted by humans. However, Judy provides

a web page that reads that same XML and provides a user-friendly visualization that reveals itself to be very useful.

4.5 Jumble

Jumble is released under the open-source GPL license. The last version is 1.3.0 and provides support for Java 8 and was released in May 2015. The minimum Java version Jumble supports is version 6, and for unit testing it is JUnit4. There's a plugin available for integration with the Eclipse IDE as well as a command line interface. Jumble only allows single class selection, although one or more test cases can be selected. Jumble generates mutants automatically, but they are not available for selection or even preview. Only method-level mutation operators are supported, and two of those are always active, namely the Conditional and Binary Arithmetic Operations operators. The other operators, namely Increments, Inline Constants, Class Pool Constants, Return Values and Switch Statement, can be activated and de-activated on demand. Jumble works by mutating byte-code and doesn't have any way to reduce the impact of equivalent mutants. As an output, Jumble provides a simple summary presenting the number of generated mutants, mutation score, and the total runtime duration. An option to generate a CSV file with more detailed data is also available. Despite having quite long release cycles, Jumble is still under active development.

4.6 Jester

Jester was developed by I.Moore in 2000 and stops having updates after only five years in 2005, so it is no longer under active development. Jester is released under a custom open-source license, uses the old JUnit3 unit tests framework and the maximum Java version support is the, also older, version 6. The only way to interact with Jester is through the command line. Jester generates mutants automatically and allows the selection of multiple classes and test cases. The tool provides only method-level operators that can not be chosen. Jester works at source code level and does not provide any cost reducing strategy or support for any equivalent mutants handling strategy. The execution output is in XML format.

4.7 MAJOR

Major mutation framework is still in under active development. At the time of writing, its version was 1.3.5, released in July 2019 and supports Java up to version 7, along with the JUnit4 unit testing framework. MAJOR uses ant for configuration, allowing classes and test cases to be selected. Major comes with its own proprietary language, Major Mutation Language (MML), which can be used to define the desired mutant operators [53]. It supports byte-code mutation combined with MSG.

4.8 Bacterio

Bacterio is a commercial tool that uses mutation to test Java applications. According to the authors, Bacterio uses mutation at the bytecode level and supports strong mutation, BB-Weak/1, BB-Weak/N, functional qualification, and flexible weak mutation. Because each mutation type has a varied effectiveness-to-efficiency ratio, testers have additional options in their testing initiatives. In its newer version, Bacterio brings the mutant schema technique, which allows reducing mutants storage needs and speeds their execution, making Bacterio faster and more efficient than in previous versions. Bacterio also lets testers run mutants on multiple computers at the same time, as long as all of the computers are on the same network. Bacterio works by generating several copies of the original application and introducing faults in each one. Then, test suites with JUnit or UISpec test cases are executed against both versions of the program, the original one and the mutated ones. If a test suite is unable to find faults in the original program but is able to detect the faults introduced into the mutant versions then the test suite is adequate and kills a high percentage of mutants.

Chapter 5

Design of the Experiment

The previous chapters provided the necessary background for mutation testing, its integration into JVM environment and existing tools for automated mutation testing of Java programs. This chapter presents the settings of the experiment, by detailing the research questions, the followed procedure and the design of the experiment.

5.1 Research Questions

Q1: How efficient the studied tools perform in terms of real-fault detection?

Since mutation testing is a popular and widely-used technique in research (Papadakis et al. [68]), it is mandatory to ensure that mutation testing tools are powerful and do not bias research results. Therefore, this question regards the effectiveness of the studied tools in terms of their ability to reveal real faults.

Q2: What is the relative cost of using mutation testing with the tools under consideration and what tool or tools perform better?

As discussed before, a major concern about mutation testing is the cost of its application since is considered to be an expensive technique requiring high computational resources and time. Identifying equivalent mutants is also a major concern, although outside the scope of our study. Answering this question can help both testers and researchers understand the trade-offs between cost and efficacy. This analysis will also capture the cost differences between the tools. We will only be looking at mutation score execution time.

Q3: Which of the studied tools are able to best assess the efficacy of test suites?

The goal of this question is to assess the capability of the tools to kill the generated mutants. *"A test suite is considered mutation-adequate if its mutation score is 100%"*. Higher mutation scores mean that more faults were detected, which then means the better the test suite is. Answering this question will help practitioners assess which tool may best assist them improving their test suites.

5.2 Test Subjects

The experiment was done by using a set of large real-world programs belonging to the Defects4J database (Just et al 2014 [55]) version 2.0. Defects4J is a *"benchmark set of reproducible real faults mined from source code repositories"*. It contains real faults that were meticulously found and isolated utilizing the version control and bug tracking systems of several open source projects. The benchmark provides a buggy and fixed software version, as well as at least one test case that reproduces the defective behavior, for each of the defects. Table 5.1 shows the name of the projects utilized in the experiment as test subjects.

5.3 Test Suites

To simulate the mutation based test process, we generate several test suites by using two state-of-the-art test generation tools, namely EvoSuite 1.0.3 (Fraser and Arcuri 2011 [40]) and Randoop 3.0.8 (Pacheco and Ernst 2007 [32]).

For each fault, we run EvoSuite and Randoop on the fixed version of the project to generate the test suites for the modified classes so that the corresponding buggy version can be fixed. 10 test suites for each fault were generated, 5 with EvoSuite and 5 with Randoop, resulting in a total of 740 test suites.

Identifier	Project Name	# Bugs	# Active Bugs	# Triggering Bugs	# Used Bugs
Chart	jfreechart	26	26	22	16
Compress	commons-compre:	47	47	30	25
Lang	commons-lang	64	62	36	25
Time	joda-time	26	25	19	8
		163	160	107	74

Table 5.1: Subjects

5.4 Mutation Testing Tools Selection

Over the years, several mutation tools have been developed by researchers and practitioners. However, all those tools are different in some way. They vary substantially in terms of fault models and performance. Those traits are sometimes overshadowed by more pragmatic considerations, such as Java version compatibility, unit tests frameworks support, integration with the development environment, and the list goes on. For our experiment, we intend to compare mutation tools for the Java programming language based on a variety of characteristics in order to expose which ones perform the best and to provide empirical evidence that others may use to facilitate their own tool selection.

Different literature shows that comparing mutation tools is a complex process, and there are no formal way for carrying out a tool comparison. However, most of the authors adopt a some systematic approach: a list of criteria (quantitative and qualitative) and a comparison method are proposed. Most of the time, the conclusions are based on how the tools are used on a set of case studies according to the method. The tool selection process was three-fold. First, a list of tools was compiled based on research publications, that is, a list of tools that had already been subject of comparative investigations. Second, the tools that were identified were downloaded, installed, and tested once all of those steps were completed. Third, criteria were used, and tools that conformed with those criteria were the only ones considered for selection.

The chosen criteria for selection was based on two aspects: existing work on Java mutation testing tools comparisons and a reflection regarding tool usages. The first was the subject of Chapter 3. The second will be the subject of this Chapter, and follows a framework proposed by Amalfitano et al. [70].

To evaluate tool usages we observe those characteristics that are important principles to be considered as comparison criteria. Given the high cost of mutation analysis, **efficiency** is a very essential consideration. Aspects that speed up the execution and the process of generating mutants are deemed vital for this study as it uses real-world programs that create a large number of mutations, and as a result, the criteria must include such features. **Usability** encompasses a number of facets, including the following: a tool may be simple to install, or it may not be; the presence of documentation, and the quality of that documentation, can shorten the learning curve; compatibility is frequently a deciding factor for the mutant generation; and the capacity to support all Java features is also an essential quality. Also, it's possible that a tool's individual mutation analysis stage may contain characteristics that would make the entire process more efficient.

All these important characteristics can be observed in the followed framework ([70]), which defines a model that "describes each tool along five dimensions, each one with one or more attributes". Some of the attributes considered were already used as comparative parameters by others researchers but a set of 11 novel attributes were introduced for a total of 21 attributes across 5 dimensions. From those 5 dimensions, 4 of them form the key characteristics used in the tools selection, namely:

1. **Version**, which includes License, Release Version and Release Year. These are fundamental attributes when selecting tools to integrate in the software development process since they represent part of the confidence companies need to formulate about a tool. It also helps check tool compatibility with the subjects.
2. **Deployment**, which includes Java version, Build-Tool Integration and Testing Framework. These form the core attributes a tool must fulfill to be consider. Tools with older or incompatible Java versions must be automatically discarded. Also, the capability to integrate the tool in the build/test chain is crucial for its usage in modern practices like continuous integration and continuous testing.

3. **Mutation Process**, which includes, among others, Test Selection. Automation is paramount when considering to use a tool as part of the modern practices in the software development life cycle so the capability to automate the test selection is a must.
4. **User-centric features**, which includes, among others, Produced Outputs and Documentation Quality. These are essential to assess how much effort engineers or testers will need to interpret the tools results - higher effort means lower productivity, and how easy it will be to integrate and efficiently use the tools. A tool with poor to none documentation must be avoided since it will result in constant delays in all phases of the process whenever a problem occurs and needs to be fixed.

According to the framework authors Amalfitano et al., it is desirable to select a mutation tool that is actively maintained and evolving. This increases the chances of using a state-of-the-art approach and timely resolving questions and issues. As so, only tools still under active development should be considered for experimental evaluation. These are MuJava, Judy, Jumble, MAJOR and PIT. Javalanche and Jester tools are no longer under active development so they were discarded. MuJava, being a graphical user interface tool (GUI), can not be automated to run tests on generated mutants and therefore is not considered. The Jumble tool was tried but the execution time even for small projects was too long and provided only very limited output analysis, so the tool was also not considered for the experimental evaluation. Judy was also discarded due to the lack of documentation which made it extremely difficult to assess set up and assess.

As the result of this process, only two tools were selected for experimental evaluation – PIT and MAJOR. The versions used were PIT 1.9.3 and MAJOR 1.3.5.

5.5 Analysis Procedure

The main goal of the tools evaluation is to compare the behavior of the tools. For the experiment, the default settings were used since specific configurations could bias the results considering the fact that the selected tools use different mutation operators and provide different capabilities. Existing research and experiments already explore some of these tools specific capabilities in detail.

Overall, the following procedure was used:

1. Each generated test suite was executed against the correspondent buggy version and went through a fixing process that consisted on removing flaky tests or tests that broke the mutation testing execution process.
2. For each generated test suite, the triggering tests were identified. Because a test suite must contain at least one test case that fails when executed against the buggy version, test suites without any triggering tests were dropped.

3. For each tool, run two (one generated with EvoSuite and one generated with Randoop) of the fixed test suites against the buggy versions of the projects to calculate the mutation score and assess the fault revealing capability of the tool.

Table 5.1 presents for each test subject the number of bugs in the set, the number of active bugs, the number of triggering bugs and the number of bugs remaining after the fixing process mentioned before and that form the set of 74 bugs used in the experiment. In total, 296 successful mutation testing executions are required.

5.6 Experiment Environment

The experiment was carried out on a personal Mac Os computer with the following configuration: a 2.5 GHz Quad-Core Intel Core i7 with 16GB RAM. Docker was used to run a linux Ubuntu 20.04 virtual environment that comprises all the tools needed to execute the experience, including custom bash scripts and the scripts provided by the Defects4j project. Defects4j out of the box only supports Major mutation tool so it was also extended to support PIT.

This environment was extremely limiting in terms of available computation resources and had a major impact on the time required to execute the whole experiment.

Chapter 6

Discussion

This section presents and discusses the results of the experiment according to the research questions under study. A complementary analysis was also conducted, and consisted of investigating which mutation operators were coupled to the faults the respective mutants were generated for.

6.1 Experimental Results

This section summarizes the experimentation results for each of the raised questions.

Q1: How efficient the studied tools perform in terms of real-fault detection?

It is essential to make certain that the tools used for mutation testing are effective and do not introduce any bias into the research results "(Papadakis et al. [68])". Since mutation testing is a common and widely used method in research, it is imperative that this standard be met. Because of this, the question at hand concerns the efficiency of the tools that have been investigated in terms of their capacity to expose real faults. To accomplish this, a mutation testing experiment was conducted against the 74 buggy versions of all our subjects (see table 5.1). The results are presented in the figure 6.1:

Chart		
Version	MAJOR	PIT
1	✓	✓
2	✓	✓
5	✗	✗
6	✓	✓
8	✓	✓
11	✓	✓
12	✓	✓
16	✓	✓
17	✓	✓
18	✓	✓
20	✗	✗
21	✓	✓
22	✓	✓
23	✓	✓
24	✓	✗
25	✓	✓
26	✓	✓

Compress		
Version	MAJOR	PIT
1	✓	✓
2	✓	✓
3	✓	✓
6	✗	✗
7	✓	✓
8	✓	✓
9	✓	✓
11	✓	✓
14	✗	✗
15	✗	✗
16	✓	✓
17	✓	✓
19	✓	✓
24	✗	✓
27	✓	✓
31	✗	✗
34	✓	✗
35	✗	✓
39	✗	✗
41	✓	✓
42	✗	✗
44	✓	✓
45	✓	✓
46	✓	✓
47	✓	✓

Lang		
Version	MAJOR	PIT
1	✓	✓
5	✓	✓
16	✓	✓
18	✓	✓
19	✓	✓
22	✓	✓
23	✓	✓
24	✓	✓
27	✓	✓
28	✓	✓
32	✓	✓
33	✓	✓
35	✓	✓
36	✓	✓
37	✓	✓
41	✓	✓
45	✓	✓
46	✓	✓
49	✓	✓
52	✓	✓
54	✓	✓
57	✓	✗
58	✗	✓
62	✓	✓
65	✓	✗

Time		
Version	MAJOR	PIT
1	✓	✓
2	✓	✓
5	✓	✓
13	✓	✗
15	✓	✓
22	✗	✗
24	✓	✓
27	✓	✗

Figure 6.1: Tools Fault Revealing Capability Matrix by Subject

By analysing the data in the above matrix (6.1), we can see that both tools were able to reveal the majority of the faults. However, results vary by target Subject. We can also observe that there are several cases where the faults that one tool was unable to reveal matches the ones the other tool also fail to reveal.

A look at the charts below (6.3 shows that:

- for the subject Chart MAJOR were able to reveal 88% of the faults and PIT 82%;
- for the subject Compress MAJOR revealed 66% while PIT revealed 74%;
- for the subject Lang MAJOR revealed 92% while PIT revealed 96%
- for the subject Time MAJOR revealed 88% while PIT revealed 63%

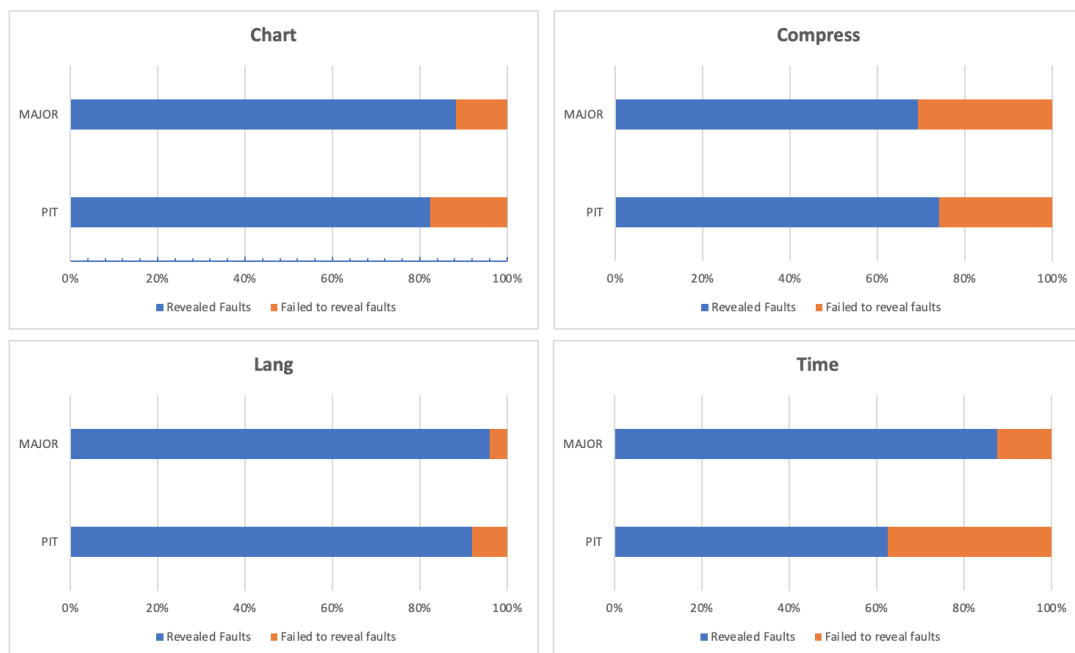


Figure 6.2: Tools Fault Revealing Capability by Subject

Overall, except for the subject Compress, MAJOR was able to reveal more faults than PIT.

Q2: What is the relative cost of using mutation testing with the tools under consideration and what tool or tools perform better?

The major problem with mutation testing is the cost of its application. It is considered to be an expensive technique, requiring high computational resources and time. Answering this question can provide useful information to both testers and researchers regarding the trade-offs between cost and effectiveness. Also, this analysis will better reflect the differences between the tools from the cost perspective.

As mentioned in the experiment design (see 5.1), we only looked at the mutation testing execution time (in minutes) to answer this question. Due to constraints in the experiment environment other metrics like CPU and consumed memory were not possible to measure so this result should be read with caution.

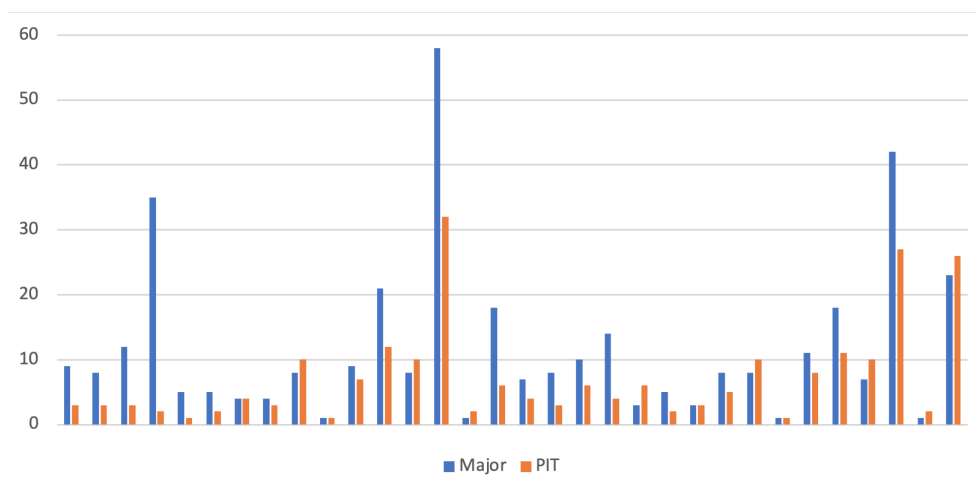


Figure 6.3: Mutation Testing Execution Time Tools Comparison

Considering the chart results, where each couple of bars represent one execution of each tool, we can see that PIT clearly behaves better in terms of execution time, which can be of maximum importance since this is not the only task in the mutation analysis process. Activities like equivalent and redundant mutants identification are usually manual tasks that add a large amount of time that cannot be ignored when measuring the cost of mutation testing. This has the potential to make its adoption infeasible. One other important observation that we can take from this experiment is that despite the significant advantage of PIT over MAJOR, in general, both present high execution times and some spikes that can be worrisome to the whole process which may lead companies rejecting mutation testing in their development cycle.

Q3: Which of the studied tools are able to best assess the efficacy of test suites?

The effectiveness of a test suite is known by its **mutation score**, which is calculated by dividing the number of killed mutants with the total number of generated mutants minus the number of equivalent mutants. But since tools can't find equivalent mutants yet [39], the real score is figured out by dividing the number of mutants killed by the total number of mutants generated.

The mutation score is supposed to help the programming teams develop more effective test suites, as the mutations tries to mimic typical programming errors.

Let's start by looking at the aggregated results from our mutation analysis executions. Table 6.1 shows, for each of our subjects and per tool, the number of generated mutants, the number of killed mutants and the correspondent mutation score. We can see that MAJOR and PIT achieved the same mutation score (49%) for the Chart subject while for the other three subjects, MAJOR beat PIT in Compress (55% against 40%) and PIT beat MAJOR in Lang (68% against 62%) and in Time with 68% against 56%. However, the overall mean result was 56% for both tools which may leads us to conclude that, in an aggregated view, no tool subsumes the other. But, if instead

of the mean we calculate the median, then PIT achieves a better score of 59% against the same 56% from MAJOR. This means that PIT behaves often better than MAJOR.

Subject	MAJOR				PIT			
	# Generated Mutants	# Mutants Killed	Mutation Score	# Revealing Mutants	# Generated Mutants	# Mutants Killed	Mutation Score	# Revealing Mutants
Chart	9582	4720	49%	98	51190	25161	49%	296
Compress	13648	7556	55%	170	79031	31339	40%	546
Lang	33200	20532	62%	337	137764	93363	68%	907
Time	9167	5143	56%	12	24438	16603	68%	25
	16399	9488	56%	617	73106	41617	56%	1774

Table 6.1: Mutation Aggregated Results Comparison

One other aspect that we can easily observe is that PIT generates way more mutants than MAJOR (usually 4 to 5 times more). What does this mean in terms of tool efficiency? In terms of mutants kill-ability, PIT also kills 4 to 5 times more mutants than MAJOR, so we conclude that this characteristic of PIT doesn't affect the efficiency of the tool in what killing mutants concerns. However, when looking at the capacity of the tool to reveal mutants, MAJOR with 1/3 of revealing mutants, when compared with PIT, is able to reveal more faults (as seen in the answer to Q1).

Now, let's observe the mutation execution results for each of the tools per subject:

Chart

Starting with Chart, we know from our previous analysis that the mean mutation score for MAJOR and PIT is 49% (the same for both tools). However, the median for PIT is 48% while for MAJOR remains 49%. This means that, for this subject, MAJOR behaves often better than PIT, although just marginally.

Compress

For Compress, MAJOR achieved a significant better mean mutation score when compared to PIT (55% against 40%). However, using median, results change also significantly with PIT raising its mutation score to 48% while MAJOR lowers its score to 53%. Although MAJOR still behaves better than PIT the difference is considerably lower.

Lang

The results obtained by running mutation tests with the subject Lang shows PIT (68%) scoring better than MAJOR (62%), again using mean. Using median, we observe the opposite we saw in Compress. In this case, MAJOR recovers from 62% to 64% while PIT loses ground and lowers to 65%.

Timer

Finally Timer. In terms of mean, PIT wins with 68% against MAJOR's 56%. In this case the use of the median doesn't significantly change the results. PIT conserves its 68% while MAJOR recovers two percentage points and raises its mutation score to 58%.

6.2 Mutation Operators - A Complementary Analysis

As we already know, mutation analysis weakness is the large number of mutants that generates and to deal with the equivalent mutants problem. This problem, as well as some of the techniques researchers are proposing to overcome it, were discussed previously. One of these techniques is **mutation selection**. Mutation selection refers to the problem of choosing, among a large number of mutants, the ones (few is better) that should be used by the testers. In view of this, we investigate the **number of mutants generated by each tool's different operators for each fault**. By doing this we can observe which (if any) operators are more coupled with the faults the mutants were generated to.

Since Java mutation tools provide different sets of mutation operators or, sometimes, the same operators but with different names and descriptions it's almost impossible to make a direct comparison between tools. This lack of standardization forces us to do this specific analysis of the tools in isolation.

PIT

The following charts (figure 6.4) shows, for each of the subjects, the number of mutants each mutation operator generated mutants to and that were able to reveal the faults. By aggregating these values by subject we can analyse what mutation operators are more effective and if there is a trend or not. A first look shows that there are a significant number of operators that have no contribution in killing mutants, while there are a few that can clearly be identified as the ones responsible for the majority of killing the mutants that are coupled with the faults.

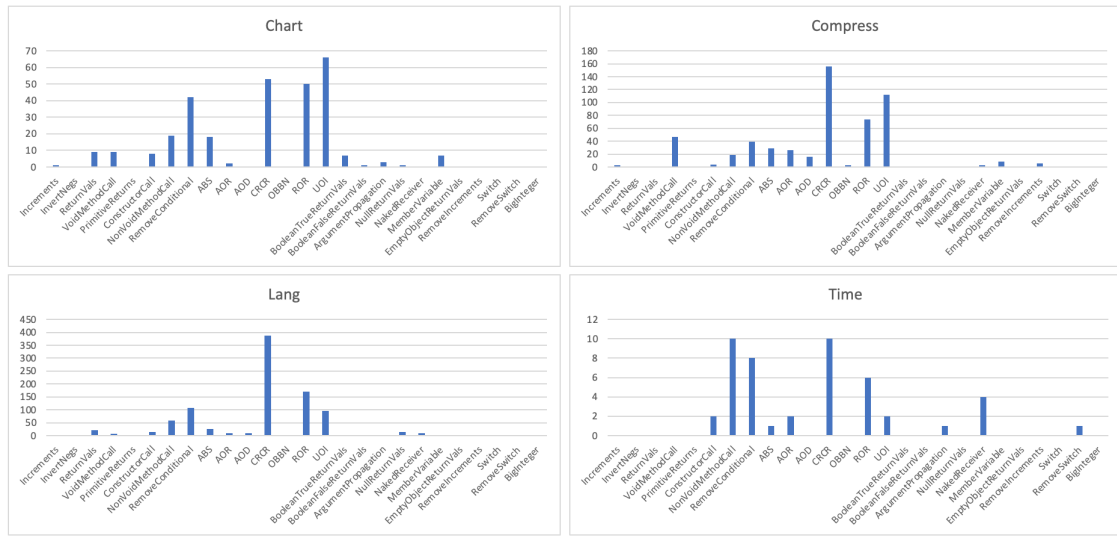


Figure 6.4: PIT Fault Revealing Mutation Operators by Subject

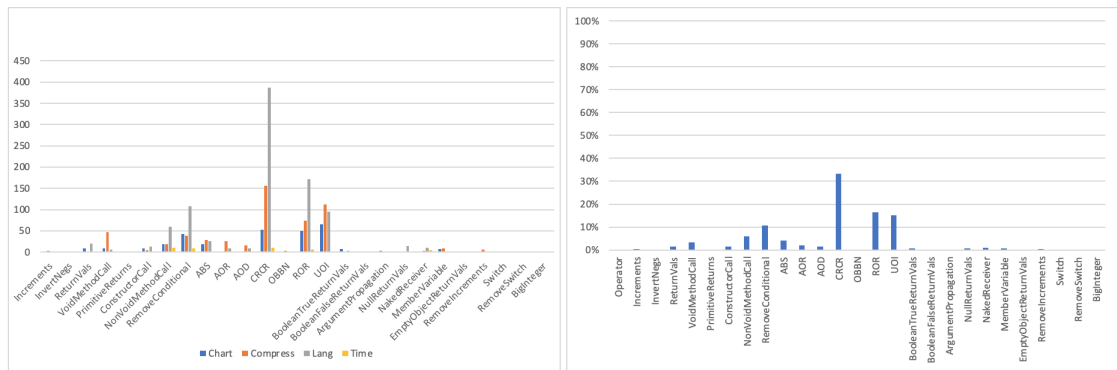


Figure 6.5: PIT Fault Revealing Mutation Operators

Zooming out a little bit on this values (see figure 6.5) the trend becomes even more clear. We can observe that just 5 of PIT's mutation operators (CRCR, ROR, UIO, RemoveConditional and NonVoidMethodCall) represent 81% of the operators responsible for killing the mutants related with the faults they were generated for. We also found that 10 operators (Increments, InvertNegs, PrimitiveReturns, BooleanFalseReturnVals, ArgumentPropagation, EmptyObjectReturnVals, RemoveIncrements, Switch, RemoveSwitch, BigInteger), in the context of our analysis, were completely useless, suggesting that their removal may help improve the computational cost of the overall mutation testing process.

MAJOR

We followed the same procedure to analyse MAJOR. So, the following data (figure 6.6) shows, for each of the subjects, the number of mutants each mutation operator generated mutants to and that were able to reveal the faults. Again, we can analyse what mutation operators are more effective and if there is a trend or not. The same as for PIT, also for MAJOR there are a significant number of operators that has low no none contribution in killing mutants, while there are a few that are easily identified as the ones responsible for the majority of killing the mutants that are couple with the faults.

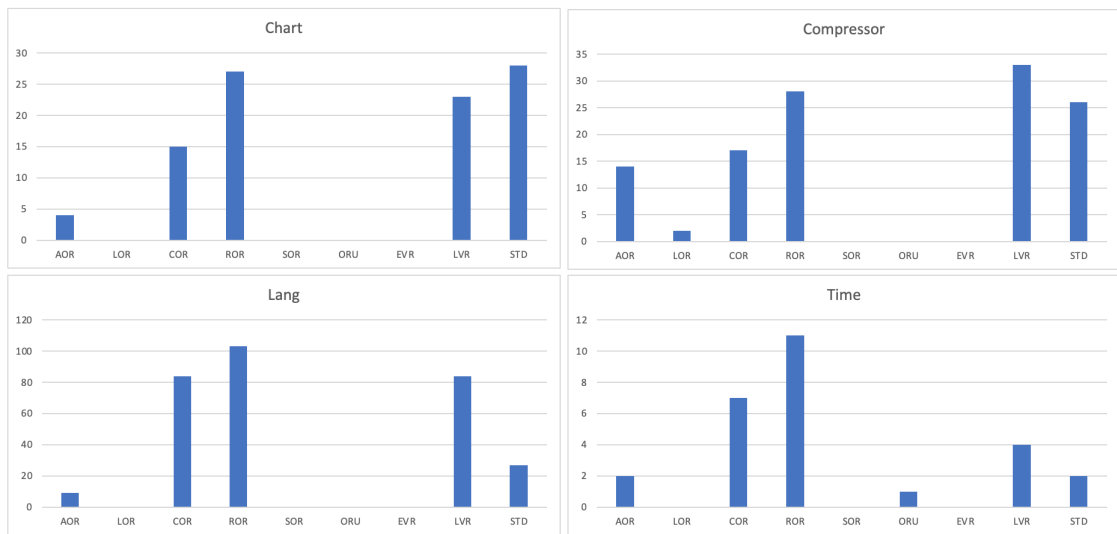


Figure 6.6: MAJOR Fault Revealing Mutation Operators by Subject

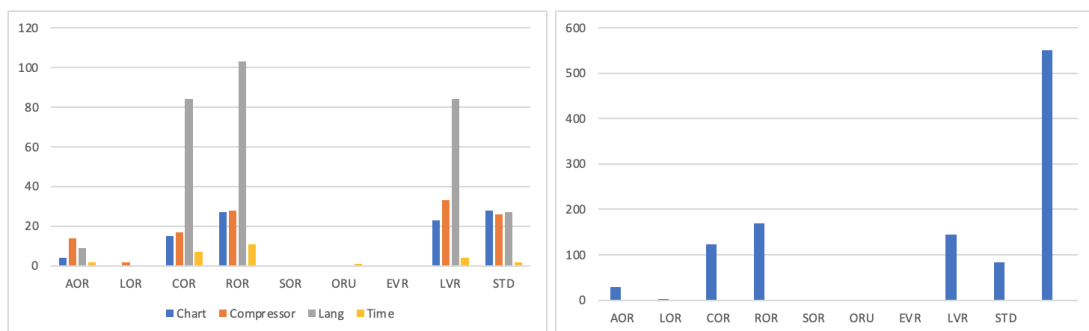


Figure 6.7: MAJOR Fault Revealing Mutation Operators

Zooming out again (figure 6.7), we can easily state that only 4 of MAJOR's mutation operators (COR, ROR, LVR, STD) represent 94% of the operators responsible for killing the mutants related with the faults they were generated for. AOR with 5%, LOR ORU with 1% closes the picture. In these case 2 of the operators were useless (SOR and EVR).

In conclusion, this analysis tell us that most probably will be a good idea to use mutant selection as way to reduce the cost of mutation analysis by executing the tools with most relevant mutation operators and the iterate on while improving the adequacy of the test suites.

6.3 Summary

The results suggest that both tools are able to reveal the majority of the subjects existing faults, with MAJOR being slightly better. In terms of computational cost, and only looking at the mutation testing execution time, PIT showed better results than MAJOR. However, the execution duration is considerably high for both tools. Finally, relative to mutation score, a brief analysis on the results shows that no tool is able to subsume the other. MAJOR gets better results with Compress but loses in the other three subjects. In general, we observe that the difference in the mutation score between the tools for each of the bugs is low and that no tool beats the other in all versions of the same subject. This emphasise the need to build a reference mutation testing tool that will be capable and strong enough of at least subsuming the existing mutation testing tools. Finally, our complementary analysis suggests that we can use mutation selection technique to significantly increase the overall effectiveness of the tools: fault revelation ability, mutation score, execution time and, as a consequence, less manual effort dealing with the equivalent mutants problem.

Chapter 7

Conclusion

Software testing means evaluating software to check whether it can perform the tasks it is supposed to perform or not. There are many different approaches to software testing. But all of them focus on how to make sure an application does not malfunction. Software testing reduces bugs, which in turn reduces additional costs and advances the performance of the software. Software testing methodologies and the process of developing end-to-end test cases have evolved drastically with the advent of new technology and the evolution of software itself. However, when software evolves, its complexity is also evolving, introducing challenges to the testing process. To deal with this issue, testing techniques must evolve along and adapt to target this increasing complexity, and help engineering teams deliver high quality software. But, to deliver high quality software, the same engineering teams must develop adequate test suites. Hence, a natural question arises: how can teams evaluate and improve the adequacy of their tests?

One such technique is Code Coverage. It helps finding out how much of our source code is actually being tested and assess the quality of test suites. However, there is no silver bullet when it comes to code coverage. Even tests with a high percentage of coverage may be ineffective if important components of the application are not tested. So, what other technique can be used to overcome this known code coverage weakness and help improve test suites adequacy?

To this end, a new testing technique called mutation testing, was proposed back in 1970s by DeMillo et al.[2]. Mutation Testing has been considered as a powerful technique for evaluating the quality of test suites. Mutation testing and Mutation testing tools are an increasing subject of research. As a consequence, this increases the search for tools that are reliable, effective, and robust when it comes to mutation testing. Some tools are now mature and robust, so the emerging question is if their effectiveness is sufficient enough for mutation testing to be adopted and how can companies easily evaluate them.

7.1 Process and Results

In this work, we conducted a controlled experiment on real world open source projects, with real faults, to evaluate the behavior of two of the most used tools in research, namely PIT and MAJOR. We provided a comparison analysis, based on the results gathered during experimentation, that can help students, researchers and practitioners evaluate which tool may best fit their needs. Overall, the results showed that none of the tools were able to clearly subsume the other.

Essentially, the experiment was conducted in two different moments:

1. Generating the artifacts needed for the experiment

The process began by selecting a set of subjects representative of real-world open source projects. For that, we chose programs from version 2.0 of the Defects4J benchmark (Just et al 2014 [55]). This is a well-known database and system that provides real faults to enable repeatable software testing researchs. Then, two, also well-known, tools for automated generation of test suites were selected, namely, EvoSuite (Fraser and Arcuri 2011 [40]) and Randoop (Pacheco and Ernst 2007 [32]). These tools were then executed 10 times (5 each tool) on every fixed version of the correspondent buggy version for a specific subject, resulting in a total of 740 test suites. Next, these test suites were executed and fixed (or eliminated), making sure that all tests passed on the fixed version and that at least one test failed on the buggy version. The intent was to settle on just 4 test suites for each version, resulting in a total of 296 test suites mapped to 74 bugs under analysis. These represented the universe of mutation analysis that had to be performed in order to answer the formulated research questions. Other artifacts needed were the creation of batch and python scripts to automate some of the tasks executed as well as customizing Defects4J to support PIT. By default, Defects4J only supports MAJOR.

2. Execute mutation testing analysis

Each Java mutation testing tool under analysis, namely MAJOR and PIT, were executed for each version of the correspondent subjects. This means that 296 mutation testing analysis were executed and its outputs processed in order to extract and consolidate the information for further analysis. After consolidation, the results were interpreted in Excel to get to the conclusions presented here.

Regarding their fault revelation capability, both tools presented satisfactory or good results, being able to reveal the majority of the faults. MAJOR was able to achieve better results than PIT in 75% of the subjects.

In terms of mutant-killability both tools behave the same with a mean score of 56% of the generate mutants being kill. We also calculate the median and observe that PIT achieve slightly better results (59% against the same 56% of MAJOR), meaning that PIT "wins the duel" more often.

Regarding the complementary study, the result's analysis showed that a reduced number of mutation operators are responsible for killing the majority of the mutants. For PIT, 5 mutation operators are responsible for 81% of the killings, while for MAJOR we just need 4 mutation operators to kill the generated mutants. This means that the usage of a technique called *mutation selection* to allow mutation testing execution only with the identified set of mutation operators of each tool, most probably, increases the effectiveness of the tools and decreases the amount of manual work needed to deal with equivalent mutants.

As verified over the years by several researchers, mutation testing is known to have a huge computational cost, something that has been an industry adoption blocker till now. In this matter, the study showed that PIT generally consumes less time doing mutation analysis, despite the fact that it generates 4 to 5 times more mutants than MAJOR. Nevertheless, both tools still require significant time to execute. When these long execution times are combined with the significant amount of manual effort required to solve the equivalent mutant problem, We believe there is still work to be done to improve the quality of mutation testing.

7.2 Future Work

As stated previously, this experiment employs the default settings of the tools and intends to provide researchers, students, and practitioners with a sense of how well these tools perform and what they are capable of out of the box. This gives them a straightforward path with just enough knowledge to begin researching and investigating deeper. In accordance with this strategy, a future investigation will follow to delve deeper into the capabilities of the researched tools, specifically how the implementation of approaches such as *mutant selection* might enhance the effectiveness of the tools.

Bibliography

- [1] Richard G. Hamlet. “Testing programs with the aid of a compiler”. In: *IEEE transactions on software engineering* 4 (1977), pp. 279–290.
- [2] Richard A DeMillo, Richard J Lipton, and Frederick G Sayward. “Hints on test data selection: Help for the practicing programmer”. In: *Computer* 11.4 (1978), pp. 34–41.
- [3] AT Acree. “On Mutation [’Ph. D. dissertation’]”. In: *Georgia Institute of Technology, Atlanta, USA* 184 (1980).
- [4] Timothy Alan Budd. *Mutation analysis of program test data*. Yale University, 1980.
- [5] Timothy A Budd and Dana Angluin. “Two notions of correctness and their relation to testing”. In: *Acta informatica* 18.1 (1982), pp. 31–45.
- [6] A Offutt. “The coupling effect: fact or fiction”. In: *ACM SIGSOFT Software Engineering Notes* 14.8 (1989), pp. 131–140.
- [7] Richard A DeMillo, A Jefferson Offutt, et al. “Constraint-based automatic test data generation”. In: *IEEE Transactions on Software Engineering* 17.9 (1991), pp. 900–910.
- [8] A Jefferson Offutt and Stephen D Lee. “How strong is weak mutation?” In: *Proceedings of the symposium on Testing, analysis, and verification*. 1991, pp. 200–213.
- [9] A Jefferson Offutt et al. “Mutation Testing of Software Using MIMD Computer.” In: *ICPP* (2). Citeseer. 1992, pp. 257–266.
- [10] A Jefferson Offutt, Gregg Rothmel, and Christian Zapf. “An experimental evaluation of selective mutation”. In: *Proceedings of 1993 15th international conference on software engineering*. IEEE. 1993, pp. 100–107.
- [11] Roland H Untch, A Jefferson Offutt, and Mary Jean Harrold. “Mutation analysis using mutant schemata”. In: *ACM SIGSOFT Software Engineering Notes*. Vol. 18. 3. ACM. 1993, pp. 139–148.
- [12] A Jefferson Offutt. “A practical system for mutation testing: help for the common programmer”. In: *Proceedings., International Test Conference*. IEEE. 1994, pp. 824–830.
- [13] A Jefferson Offutt and W Michael Craft. “Using compiler optimization techniques to detect equivalent mutants”. In: *Software Testing, Verification and Reliability* 4.3 (1994), pp. 131–154.

- [14] A Jefferson Offutt and Stephen D Lee. “An empirical evaluation of weak mutation”. In: *IEEE Transactions on Software Engineering* 20.5 (1994), pp. 337–344.
- [15] A Jefferson Offutt, Jeff Voas, and Jeff Payne. *Mutation operators for Ada*. Tech. rep. Technical Report ISSE-TR-96-09, Information and Software Systems Engineering . . . , 1996.
- [16] A Jefferson Offutt et al. “An experimental determination of sufficient mutant operators”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 5.2 (1996), pp. 99–118.
- [17] Phyllis G Frankl, Stewart N Weiss, and Cang Hu. “All-uses vs mutation testing: an experimental comparison of effectiveness”. In: *Journal of Systems and Software* 38.3 (1997).
- [18] A Jefferson Offutt and Jie Pan. “Automatically detecting equivalent mutants and infeasible paths”. In: *Software testing, verification and reliability* 7.3 (1997), pp. 165–192.
- [19] Hong Zhu, Patrick AV Hall, and John HR May. “Software unit test coverage and adequacy”. In: *Acm computing surveys (csur)* 29.4 (1997), pp. 366–427.
- [20] Richard Sevenich and Rick Price. “Javalanche”. In: *Linux Journal* 1998.51es (1998), 5–es.
- [21] Sunwoo Kim, John A Clark, and John A McDermid. “Assessing test set adequacy for object oriented programs using class mutation”. In: *Proceedings of the 3rd Symposium on Software Technology (SoST’99)*. Citeseer. 1999.
- [22] Sunwoo Kim, John A Clark, and John A McDermid. “Class mutation: Mutation testing for object-oriented programs”. In: *Proc. Net. ObjectDays*. Net. Objects Erfurt, Germany. 2000, pp. 9–12.
- [23] Taffine Murnane and Karl Reed. “On the effectiveness of mutation analysis as a black box testing technique”. In: *Proceedings 2001 Australian Software Engineering Conference*. IEEE. 2001, pp. 12–20.
- [24] A Jefferson Offutt and Roland H Untch. “Mutation 2000: Uniting the orthogonal”. In: *Mutation testing for the new century* (2001), pp. 34–44.
- [25] Jeff Offutt et al. “A fault model for subtype inheritance and polymorphism”. In: *Proceedings 12th International Symposium on Software Reliability Engineering*. IEEE. 2001, pp. 84–93.
- [26] Yu-Seung Ma, Yong-Rae Kwon, and Jeff Offutt. “Inter-class mutation operators for Java”. In: *13th International Symposium on Software Reliability Engineering, 2002. Proceedings*. IEEE. 2002, pp. 352–363.
- [27] Boris Beizer. *Software testing techniques*. Dreamtech Press, 2003.
- [28] Michael Olan. “Unit testing: test early, test often”. In: *Journal of Computing Sciences in Colleges* 19.2 (2003), pp. 319–328.
- [29] James H Andrews, Lionel C Briand, and Yvan Labiche. “Is mutation an appropriate tool for testing experiments?” In: *Proceedings of the 27th international conference on Software engineering*. 2005, pp. 402–411.

- [30] Jeremy S Bradbury, James R Cordy, and Juergen Dingel. “Mutation operators for concurrent Java (J2SE 5.0)”. In: *Second Workshop on Mutation Analysis (Mutation 2006-ISSRE Workshops 2006)*. IEEE. 2006, pp. 11–11.
- [31] Jeff Offutt, Yu-Seung Ma, and Yong-Rae Kwon. “The class-level mutants of MuJava”. In: *Proceedings of the 2006 international workshop on Automation of software test*. 2006, pp. 78–84.
- [32] Carlos Pacheco et al. “Feedback-directed random test generation”. In: *29th International Conference on Software Engineering (ICSE’07)*. IEEE. 2007, pp. 75–84.
- [33] P. Ammann and A. Offutt. “Introduction to Software Testing”. In: 2008.
- [34] Bernhard JM Grün, David Schuler, and Andreas Zeller. “The impact of equivalent mutants”. In: *2009 International Conference on Software Testing, Verification, and Validation Workshops*. IEEE. 2009, pp. 192–199.
- [35] David Schuler, Valentin Dallmeier, and Andreas Zeller. “Efficient mutation testing by checking invariant violations”. In: *Proceedings of the eighteenth international symposium on Software testing and analysis*. 2009, pp. 69–80.
- [36] David Schuler and Andreas Zeller. “Javalanche: Efficient mutation testing for Java”. In: *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*. 2009, pp. 297–298.
- [37] Yue Jia and Mark Harman. “An analysis and survey of the development of mutation testing”. In: *IEEE transactions on software engineering* 37.5 (2010), pp. 649–678.
- [38] Huai Liu et al. “Adaptive random testing by exclusion through test profile”. In: *2010 10th International Conference on Quality Software*. IEEE. 2010, pp. 92–101.
- [39] Lech Madeyski and Norbert Radyk. “Judy—a mutation testing tool for Java”. In: *IET software* 4.1 (2010), pp. 32–42.
- [40] Gordon Fraser and Andrea Arcuri. “Evosuite: automatic test suite generation for object-oriented software”. In: *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 2011, pp. 416–419.
- [41] Jingyu Hu, Nan Li, and Jeff Offutt. “An analysis of OO mutation operators”. In: *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. IEEE. 2011, pp. 334–341.
- [42] René Just, Gregory M Kapfhammer, and Franz Schweiggert. “Using conditional mutation to increase the efficiency of mutation analysis”. In: *Proceedings of the 6th International Workshop on Automation of Software Test*. ACM. 2011, pp. 50–56.
- [43] René Just, Gregory M. Kapfhammer, and Franz Schweiggert. “Using conditional mutation to increase the efficiency of mutation analysis”. In: *Proceedings of the International Workshop on Automation of Software Test (AST)*. May 2011, pp. 50–56.

- [44] René Just, Franz Schweiggert, and Gregory M. Kapfhammer. “MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler”. In: *Proceedings of the International Conference on Automated Software Engineering (ASE)*. Nov. 2011, pp. 612–615.
- [45] Simona Nica, Rudolf Ramler, and Franz Wotawa. “Is mutation testing scalable for real-world software projects”. In: *VALID Third International Conference on Advances in System Testing and Validation Lifecycle, Barcelona, Spain*. 2011.
- [46] Jeff Offutt. “A mutation carol: Past, present and future”. In: *Information and Software Technology* 53.10 (2011), pp. 1098–1107.
- [47] René Just, Gregory M Kapfhammer, and Franz Schweiggert. “Using non-redundant mutation operators and test suite prioritization to achieve efficient and scalable mutation analysis”. In: *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*. Nov. 2012, pp. 11–20.
- [48] René Just, Gregory M. Kapfhammer, and Franz Schweiggert. “Do redundant mutants affect the effectiveness and efficiency of mutation analysis?” In: *Proceedings of the International Workshop on Mutation Analysis (Mutation)*. Apr. 2012, pp. 720–725.
- [49] Mickaël Delahaye and Lydie Du Bousquet. “A comparison of mutation analysis tools for java”. In: *2013 13th International Conference on Quality Software*. IEEE. 2013, pp. 187–195.
- [50] Lin Deng, Jeff Offutt, and Nan Li. “Empirical evaluation of the statement deletion mutation operator”. In: *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. IEEE. 2013, pp. 84–93.
- [51] Lech Madeyski et al. “Overcoming the equivalent mutant problem: A systematic literature review and a comparative experiment of second order mutation”. In: *IEEE Transactions on Software Engineering* 40.1 (2013), pp. 23–42.
- [52] David Schuler and Andreas Zeller. “Covering and uncovering equivalent mutants”. In: *Software Testing, Verification and Reliability* 23.5 (2013), pp. 353–374.
- [53] René Just. “The Major mutation framework: Efficient and scalable mutation analysis for Java”. In: *Proceedings of the 2014 international symposium on software testing and analysis*. 2014, pp. 433–436.
- [54] René Just, Michael D. Ernst, and Gordon Fraser. “Efficient mutation analysis by propagating and partitioning infected execution states”. In: *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*. **ACM SIGSOFT Distinguished Paper Award**. San Jose, CA, USA, July 2014, pp. 315–326.
- [55] René Just, Darioush Jalali, and Michael D Ernst. “Defects4J: A database of existing faults to enable controlled testing studies for Java programs”. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. 2014, pp. 437–440.

- [56] René Just et al. “Are mutants a valid substitute for real faults in software testing?” In: *Proceedings of the Symposium on the Foundations of Software Engineering (FSE)*. **ACM SIGSOFT Distinguished Paper Award**. Hong Kong, Nov. 2014, pp. 654–665.
- [57] Mickaël Delahaye and Lydie Du Bousquet. “Selecting a software engineering tool: lessons learnt from mutation analysis”. In: *Software: Practice and Experience* 45.7 (2015), pp. 875–891.
- [58] Mike Papadakis et al. “Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. IEEE. 2015, pp. 936–946.
- [59] Shweta Rani, Bharti Suri, and Sunil Kumar Khatri. “Experimental comparison of automated mutation testing tools for java”. In: *2015 4th International Conference on Reliability, Info-com Technologies and Optimization (ICRITO)(Trends and Future Directions)*. IEEE. 2015, pp. 1–6.
- [60] Marinos Kintis. “Effective methods to tackle the equivalent mutant problem when testing software with mutation”. PhD thesis. Οικονομικό Πανεπιστήμιο Αθηνών. Τμήμα Πληροφορικής, 2016.
- [61] Daniel Klischies and Konrad Fögen. “An analysis of current mutation testing techniques applied to real world examples”. In: *Full-scale Software Engineering/Current Trends in Release Engineering* 13 (2016).
- [62] Jakub Mořucha and Bruno Rossi. “Is mutation testing ready to be adopted industry-wide?” In: *International Conference on Product-Focused Software Process Improvement*. Springer. 2016, pp. 217–232.
- [63] Thierry Titchou Chekam et al. “An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption”. In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE. 2017, pp. 597–608.
- [64] Rahul Gopinath et al. “Does choice of mutation tool matter?” In: *Software Quality Journal* 25.3 (2017), pp. 871–920.
- [65] András Márki and Birgitta Lindström. “Mutation tools for java”. In: *Proceedings of the Symposium on Applied Computing*. 2017, pp. 1364–1415.
- [66] Marinos Kintis et al. “How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults”. In: *Empirical Software Engineering* 23.4 (2018), pp. 2426–2463.
- [67] Ana CR Paiva et al. “Testing when mobile apps go to background and come back to foreground”. In: *2019 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2019, pp. 102–111.

- [68] Mike Papadakis et al. “Mutation testing advances: an analysis and survey”. In: *Advances in Computers*. Vol. 112. Elsevier, 2019, pp. 275–378.
- [69] Ana CR Paiva, André Restivo, and Sérgio Almeida. “Test case generation based on mutations over user execution traces”. In: *Software Quality Journal* 28.3 (2020), pp. 1173–1186.
- [70] DOMENICO Amalfitano et al. “How do Java mutation tools differ?” In: *Communications of the ACM* 65.12 (Dec. 2022). DOI: [10.1145/3526099](https://doi.org/10.1145/3526099).
- [71] Jakub Mořucha. “Current Status of Source Code Mutation Testing in JVM Environments”. In: ().

(Appendices)

Appendix A

Tables

In this Appendix we present the detailed results of our experiment. Table [A.1](#) presents for the subject *Chart*, for each of the tools (MAJOR and PIT) and per program version (Bug ID), the number of generated mutants, the number of mutants that were killed, the number of mutants that survived and respective mutation score subject. Tables [A.2](#), [A.3](#), [A.4](#) present the same results but for the subjects *Compress*, *Lang* and *Time* respectively.

Bug ID	MAJOR				PITest			
	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score
1	336	223	113	66%	2605	784	1821	30%
	343	232	111	68%	2605	783	1822	30%
2	964	581	383	60%	6863	4420	2443	64%
	943	604	339	64%	6863	4412	2451	64%
5	297	127	170	43%	1248	592	656	47%
	297	129	168	43%	1248	570	678	46%
6	13	7	6	54%	219	68	151	31%
	12	6	6	50%	219	60	159	27%
11	486	134	352	28%	2148	708	1440	33%
	486	140	346	29%	2148	736	1412	34%
12	236	93	143	39%	905	435	470	48%
	236	94	142	40%	905	457	448	50%
16	307	195	112	64%	1243	901	342	72%
	306	168	138	55%	1243	846	397	68%
17	519	252	267	49%	2003	1049	954	52%
	519	252	267	49%	2003	1084	919	54%
18	313	192	121	61%	1541	1071	470	70%
	314	195	119	62%	1541	1082	459	70%
20	16	11	5	69%	118	77	41	65%
	16	11	5	69%	118	83	35	70%
21	166	60	106	36%	905	386	519	43%
	166	61	105	37%	905	408	497	45%
22	172	91	81	53%	846	545	301	64%
	172	91	81	53%	846	529	317	63%
23	178	53	125	30%	1011	321	690	32%
	178	31	147	17%	1011	218	793	22%
24	40	35	5	88%	242	206	36	85%
	39	34	5	87%	242	190	52	79%
25	355	147	208	41%	1918	343	1575	18%
	127	40	87	31%	1918	236	1682	12%
26	515	218	297	42%	1780	807	973	45%
	515	213	302	41%	1780	754	1026	42%

Table A.1: Mutation Results Comparison: Chart Subject

Bug ID	MAJOR				PITest			
	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score
1	393	259	134	66%	1461	832	629	57%
	393	183	210	47%	1461	412	1049	28%
2	131	51	80	39%	894	211	683	24%
	139	49	90	35%	894	207	687	23%
3	130	54	76	42%	784	342	442	44%
	135	59	76	44%	784	357	427	46%
6	151	77	74	51%	698	388	310	56%
	151	71	80	47%	698	342	356	49%
7	192	132	60	69%	1072	540	532	50%
	177	88	89	50%	1072	342	730	32%
8	246	124	122	50%	1335	556	779	42%
	246	118	128	48%	1335	550	785	41%
9	131	57	74	44%	858	329	529	38%
	146	73	73	50%	858	412	446	48%
11	61	45	16	74%	346	207	139	60%
	61	46	15	75%	346	206	140	60%
14	341	201	140	59%	1778	843	935	47%
	341	169	172	50%	1778	726	1052	41%
15	259	160	99	62%	1041	640	401	61%
	255	155	100	61%	1041	630	411	61%
16	62	34	28	55%	355	175	180	49%
	62	35	27	56%	355	170	185	48%
17	578	281	297	49%	2919	1165	1754	40%
	549	237	312	43%	2919	990	1929	34%
19	226	147	79	65%	821	502	319	61%
	226	116	110	51%	821	388	433	47%
24	553	260	293	47%	2872	1095	1777	38%
	574	274	300	48%	2872	1133	1739	39%
27	583	309	274	53%	2832	1351	1481	48%
	579	273	306	47%	2832	1129	1703	40%
31	578	330	248	57%	2832	1411	1421	50%
	555	296	259	53%	2832	1318	1514	47%
34	114	82	32	72%	562	373	189	66%
	114	82	32	72%	563	355	208	63%
35	517	261	256	50%	2692	1249	1443	46%
	527	337	190	64%	2692	1499	1193	56%
39	118	85	33	72%	776	512	264	66%
	118	85	33	72%	776	508	268	65%
41	124	54	70	44%	3498	343	3155	10%
	117	43	74	37%	3498	293	3205	8%
42	309	309	0	100%	1329	850	479	64%
	309	309	0	100%	1329	909	420	68%
44	25	21	4	84%	120	102	18	85%
	25	16	9	64%	120	88	32	73%
45	540	319	221	59%	2761	1363	1398	49%
	520	282	238	54%	2761	1224	1537	44%
46	323	188	135	58%	1221	666	555	55%
	326	195	131	60%	1221	690	531	57%
47	164	60	104	37%	3658	351	3307	10%
	154	65	89	42%	3658	65	3593	2%

Table A.2: Mutation Results Comparison: Compress Subject

Bug ID	MAJOR				PITest			
	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score
1	925	479	446	52%	4406	2680	1726	61%
	925	472	453	51%	4406	2639	1767	60%
5	176	120	56	68%	852	539	313	63%
	182	122	60	67%	852	533	319	63%
7	879	488	391	56%	4093	2563	1530	63%
	879	484	395	55%	4093	2551	1542	62%
8	672	425	247	63%	1470	1014	456	69%
	672	461	211	69%	1470	1028	442	70%
16	865	481	384	56%	4035	2509	1526	62%
	865	483	382	56%	4035	2563	1472	64%
18	661	474	187	72%	1635	1247	388	76%
	655	468	187	71%	1635	1190	445	73%
19	161	105	56	65%	648	435	213	67%
	161	105	56	65%	648	434	214	67%
22	909	594	315	65%	3595	2839	756	79%
	909	605	304	67%	3595	2972	623	83%
23	218	85	133	39%	1250	495	755	40%
	218	88	130	40%	1250	533	717	43%
24	853	458	395	54%	4017	2453	1564	61%
	853	452	401	53%	4017	2424	1593	60%
27	848	451	397	53%	4004	2470	1534	62%
	848	464	384	55%	4004	2492	1512	62%
28	80	54	26	68%	353	232	121	66%
	80	54	26	68%	353	234	119	66%
33	403	254	149	63%	1758	1248	510	71%
	403	247	156	61%	1758	1236	522	70%
35	2162	1456	706	67%	8441	6124	2317	73%
	2162	1499	663	69%	8441	6127	2314	73%
36	839	457	382	54%	3967	2483	1484	63%
	839	465	374	55%	3967	2546	1421	64%
37	2162	1470	692	68%	8450	6140	2310	73%
	2162	1455	707	67%	8450	6157	2293	73%
41	400	256	144	64%	1743	1257	486	72%
	400	261	139	65%	1743	1288	455	74%
45	322	185	137	57%	1340	910	430	68%
	322	213	109	66%	1340	977	363	73%
46	195	138	57	71%	1195	714	481	60%
	195	148	47	76%	1195	766	429	64%
49	893	601	292	67%	3507	2907	600	83%
	894	612	282	68%	3507	3011	496	86%
52	133	102	31	77%	917	597	320	65%
	130	99	31	76%	917	543	374	59%
54	203	107	96	53%	725	409	316	56%
	203	107	96	53%	725	395	330	54%
57	194	120	74	62%	679	405	274	60%
	194	107	87	55%	679	345	334	51%
58	1010	575	435	57%	4610	3024	1586	66%
	1010	595	415	59%	4610	3042	1568	66%
62	449	288	161	64%	1192	922	270	77%
	427	243	184	57%	1192	721	471	60%

Table A.3: Mutation Results Comparison: Lang Subject

Bug ID	MAJOR				PITest			
	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score	# Generated Mutants	# Mutants Killed	# Mutants Survived	Score
1	413	270	143	65%	1911	1385	526	72%
	414	257	157	62%	1911	1300	611	68%
2	395	251	144	64%	1839	1315	524	72%
	394	270	124	69%	1839	1313	526	71%
5	526	198	328	38%	3257	2205	1052	68%
	501	199	302	40%	3257	2122	1135	65%
13	1303	739	564	57%	1051	898	153	85%
	1310	667	643	51%	1051	845	206	80%
15	272	191	81	70%	1292	775	517	60%
	267	182	85	68%	1292	710	582	55%
22	177	96	81	54%	976	443	533	45%
	185	107	78	58%	976	496	480	51%
24	220	125	95	57%	842	521	321	62%
	220	133	87	60%	842	568	274	67%
27	1285	753	532	59%	1051	888	163	84%
	1285	705	580	55%	1051	819	232	78%

Table A.4: Mutation Results Comparison: Time Subject

Appendix B

Charts

In this Appendix we present a set of charts comparing the achieved mutation scores by executing each of the tools (MAJOR and PIT) against our set of subjects (Chart, Compress, Lang and Time) and their corresponding program versions used in our experiment.

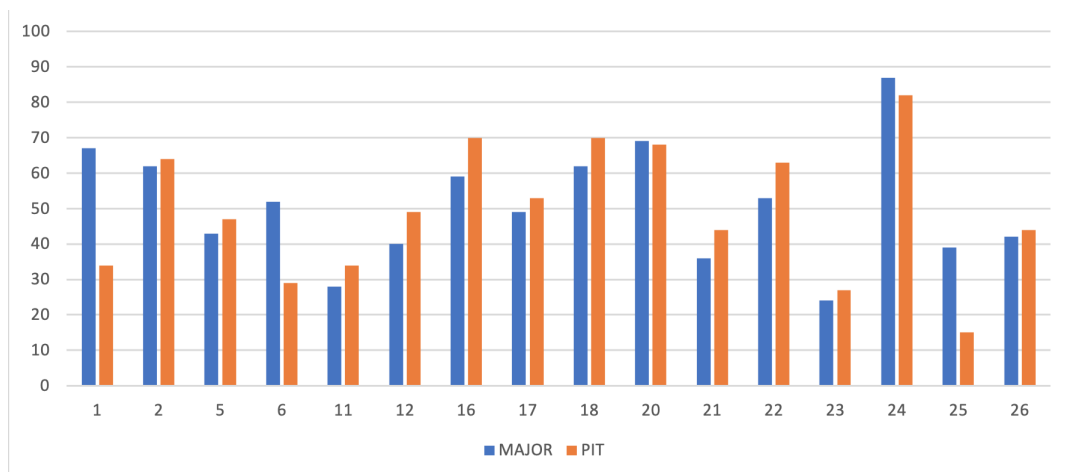


Figure B.1: Mutation Results Comparison: Chart Subject

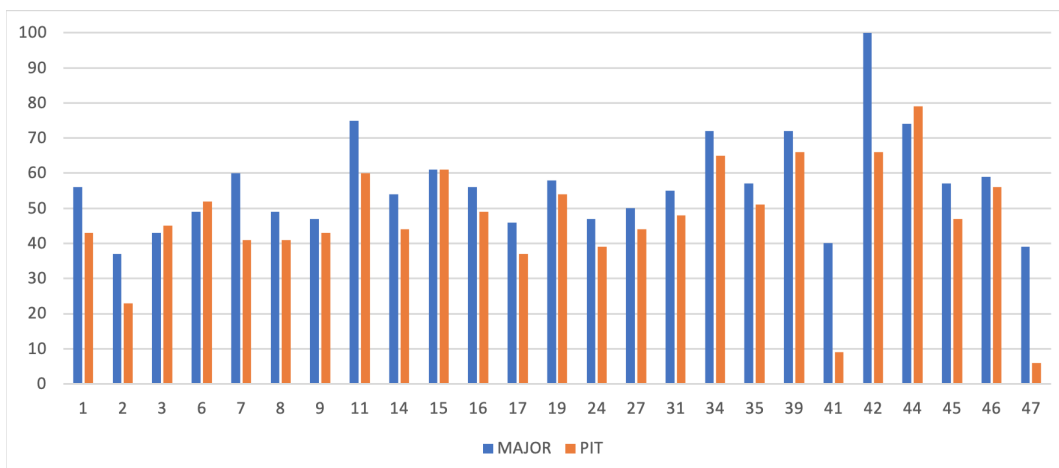


Figure B.2: Mutation Results Comparison: Compress Subject

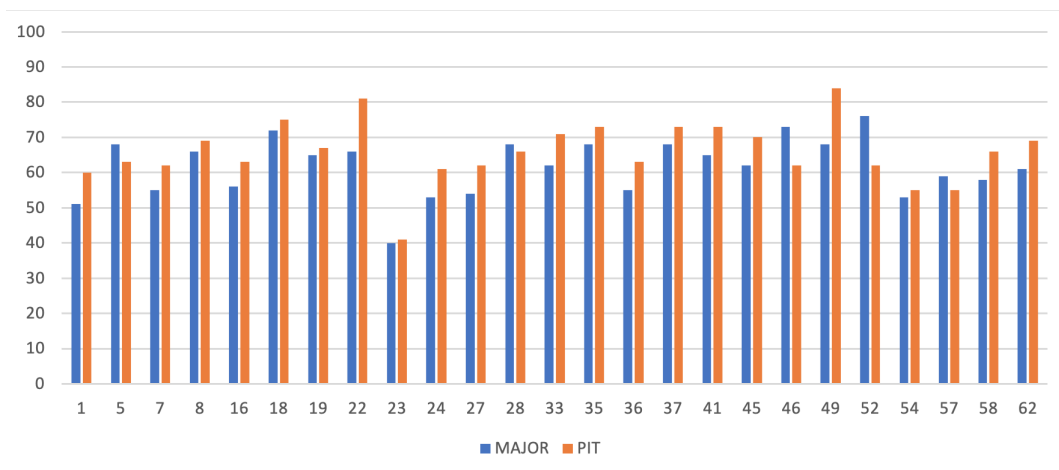


Figure B.3: Mutation Results Comparison: Lang Subject

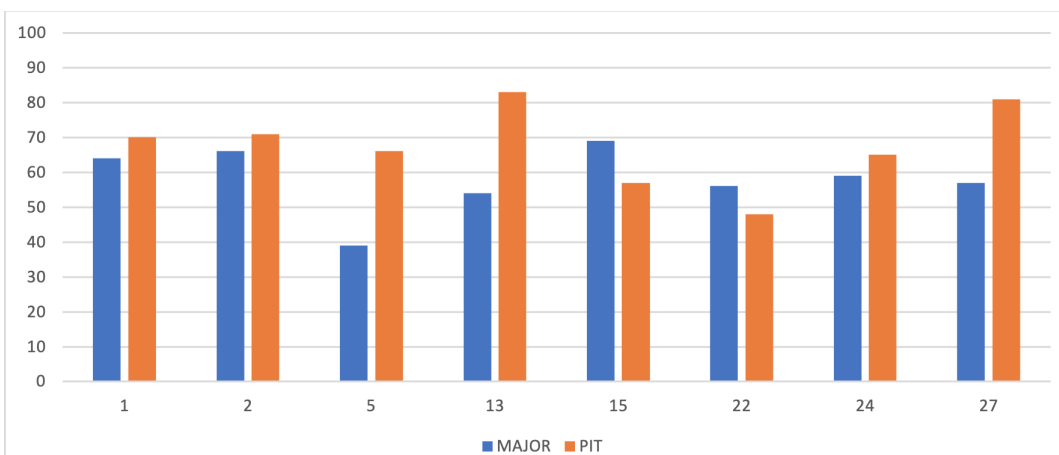


Figure B.4: Mutation Results Comparison: Time Subject